2017 届研究生硕士学位论文

分 类 号：　　学校代码：10269

密　　级：　　学　　号：51141201055



華東師範大學

East China Normal University

硕士学位论文

MASTER'S DISSERTATION

论文题目： 云计算环境中利润最大化的最优服务定价与多服务器系统配置研究

院　　系：计算机科学与技术系

专　　业：计算机应用技术

研 究 方 向：实时系统与云计算

指 导 教 师：魏同权 副教授

学位申请人：邵高原

2017 年 3 月

2017 届研究生硕士学位论文

分类号：_____ 学校代码：10269

密　级：_____ 学　　号：51141201055

華東師範大學

East China Normal University

硕士学位论文

MASTER'S DISSERTATION

论文题目：云计算环境中利润最大化的最优服务定价与多服务器系统配置研究

院　　系：计算机科学与技术系

专　　业：计算机应用技术

研 究 方 向：实时系统与云计算

论文指导教师：魏同权 副教授

学 位 申 请 者：邵高原

2017 年 3 月

Dissertation for Master's Degree in 2017

University Code: 10269
Student ID: 51141201055

East China Normal University

Title: Optimal Cloud Service Pricing and Multi-server System Configuration for Profit Maximization in Cloud Computing

Department: Department of Computer Science

Major: Computer Application and Technology

Research Direction: Cloud Computing and Real-Time Systems

Supervisor: Prof. Wei Tongquan

Candidate: Shao Gaoyuan

March, 2017

华东师范大学学位论文原创性声明

郑重声明：本人呈交的学位论文《云计算环境中利润最大化的最优服务定价与多服务器系统配置研究》，是在华东师范大学攻读硕士/博士（请勾选）学位期间，在导师的指导下进行的研究工作及取得的研究成果。除文中已经注明引用的内容外，本论文不包含其他个人已经发表或撰写过的研究成果。对本文的研究做出重要贡献的个人和集体，均已在文中作了明确说明并表示谢意。

作者签名：邵高原　　日期：2017年5月26日

华东师范大学学位论文著作权使用声明

《云计算环境中利润最大化的最优服务定价与多服务器系统配置研究》系本人在华东师范大学攻读学位期间在导师指导下完成的硕士/博士（请勾选）学位论文，本论文的研究成果归华东师范大学所有。本人同意华东师范大学根据相关规定保留和使用此学位论文，并向主管部门和相关机构如国家图书馆、中信所和“知网”送交学位论文的印刷版和电子版；允许学位论文进入华东师范大学图书馆及数据库被查阅、借阅；同意学校将学位论文加入全国博士、硕士学位论文共建单位数据库进行检索，将学位论文的标题和摘要汇编出版，采用影印、缩印或者其它方式合理复制学位论文。

本学位论文属于（请勾选）

（ ）1.经华东师范大学相关部门审查核定的“内部”或“涉密”学位论文*，于　　年　　月　　日解密，解密后适用上述授权。

（√）2.不保密，适用上述授权。

导师签名　　本人签名邵高原

2017年5月26日

* “涉密”学位论文应是已经华东师范大学学位评定委员会办公室或保密委员会审定过的学位论文（需附获批的《华东师范大学研究生申请学位论文“涉密”审批表》方为有效），未经上述部门审定的学位论文均为公开学位论文。此声明栏不填写的，默认为公开学位论文，均适用上述授权）。

邵高原 硕士学位论文答辩委员会成员名单（必须）

姓名	职称	单位	备注
张 卫	教授	华东师范大学计算机科学技术系	主席
黄新力	副教授	华东师范大学计算机科学技术系	
陆 刚	副教授	华东师范大学计算机科学技术系	

摘 要

作为提供计算资源和服务的一种高效方式，云计算越来越受欢迎。由于云计算是一种商业计算模型，因此了解其经济性至关重要。从云服务运营商的角度来看，获取高额利润是最主要的目标，主要取决于给定市场需求下的服务费用和业务成本。为了最大化利润，运营商应该了解其服务费用和业务成本的构成，以及它们是如何由用户的需求分布、应用程序的特性及多服务器系统的配置所决定。

本文研究了云计算环境中利润最大化的最优云服务定价和最优多服务器配置问题。在研究过程中，我们对利润最大化问题进行了如下建模。首先，基于用户的感知价值，构建了一个能根据服务价格的变化来反映用户需求分布且符合市场环境中用户需求定律的定价模型。然后，基于用户服务请求对延迟的敏感性，提出了一个 MR (Monetary Reward)返现奖励模型。最后，构建了一个基于服务请求响应时间的低服务质量费用补偿模型。为了方便分析，本文将多服务器系统作为一个 M/M/m 排队模型来处理，并且考虑了两种服务器功耗模式，即 idle-speed 和 constant-speed 模式。此外，本文还构建了一个动态自适应闭环控制系统模型(DALCS)，使得本文提出的研究方法能动态适应外部条件的改变。研究过程中，本文提出的模型考虑了如下因素：服务请求的数量，服务请求的紧急程度，应用环境的工作负载，多处理器集群的配置，服务等级协议(SLA)，服务质量(QoS)，用户的服务满意度，低服务质量时的费用补偿，对提交弹性服务请求用户的费用奖励，多服务器系统租赁费用，能耗费用，市场环境中用户的动态需求分布。

实验结果表明，本文提出的定价模型符合用户的需求定律。利润最大化的最优服务定价和最优的服务器工作数量及运行速度都能很好的获得。与已有的研究方案 OMCPM 及 UPMR 相比，本文构建的利润最大化模型在不同多服务器系统配置下所获得的利润均高于前两者。

关键词： 云计算，用户感知价值，利润最大化，最优定价，服务器配置，服务质量，返现奖励，动态自适应

ABSTRACT

As an effective and efficient way to provide computing resources and services to customers on demand, cloud computing has become more and more popular. Moreover, as a business computing model, understanding the economics of cloud computing becomes critically important. From the perspective of cloud service provider, profit is the most concerned aspect and mainly determined by service charges and business costs under given market demand. To maximize the profit, service provider need to understand both constructs of service charges and business costs, and how they are determined by users demand distribution, the characteristics of the applications and the configuration of a multi-server system.

This paper aims to solve the profit maximize problem which refers to optimal cloud service pricing and optimal multi-server configuration. We modeled the profit maximize problem as follows. First, we proposed a pricing model which can be used to model users demand distribution based on users perceived value. Second, based on the delay sensitivity of users service requests, a MR (Monetary Reward) model is presented. At last, we proposed a services charge penalty model for the lower service quality. The multi-server system is treated as an M/M/m queuing model, and two power consumption mode are considered, namely, idle-speed mode and constant-speed mode. Moreover, this paper proposed a Dynamic Adaption Loop Control system (DALCS) model. The DALCS model can dynamically adapt to the change of market conditions. In the study of the research mentioned above, the profit maximize problem takes the following factors into account: the amounts of service, the sensitivity of a service, the workload of an application environment, the configuration of a multi-server system, the service-level agreement (SLA), the quality of service (QoS), the satisfaction of a consumer, the penalty of a low-quality service, the cost of renting of a multi-server system, the cost of energy consumption and the users dynamic demand distribution in market environments.

The experimental results showed that the proposed pricing model meets users demand distribution. The optimal service price, optimal amounts and speed of multi-server system have obtained. Under various simulation conditions, the max profit in a unit time is higher than the existed research OMCPM and UPMR.

KeyWords: cloud computing, user perceived value, profit maximization, optimal price, server configuration, quality of service, monetary reward, dynamic adaption

目 录

摘 要.....I
ABSTRACT.....II
主要参数及含义对照表.....VI
插图和表格目录清单.....VII
第一章 引言.....1
1.1 研究背景与意义.....1
1.2 国内外研究现状分析.....2
1.2.1 云服务中固定定价策略的研究.....3
1.2.2 云服务中动态定价策略的研究.....4
1.2.3 关于云服务运营商利润最大化的研究.....5
1.3 本文的研究目标和内容.....6
1.4 本文的创新点.....8
1.5 本文的组织结构.....9
第二章 系统架构和模型.....10
2.1 典型的云计算架构模型.....10
2.2 用户模型.....11
2.2.1 用户感知价值.....11
2.2.2 用户需求分布.....12
2.2.3 用户 MR 返现奖励模型.....16
2.3 云服务运营商模型.....18
2.3.1 多服务器模型.....19
2.3.2 收入模型.....21
2.3.3 功耗模型与电费支出.....22
2.3.4 租赁费用模型.....23
2.3.5 对弹性用户的奖励模型.....23
2.3.6 低服务质量的收费惩罚模型.....24
2.4 本章小结.....26
第三章 最优服务定价与系统配置的研究.....27

3.1 问题定义.....27
3.2 分解算法.....29
3.3 求解最优服务定价与系统配置参数.....33
3.4 本章小结.....40
第四章 动态自适应闭环控制系统的研究.....42
4.1 动态自适应闭环控制系统模型概述.....42
4.2 动态自适应闭环控制系统的算法设计.....44
4.3 本章小结.....47
第五章 实验设置与结果分析.....48
5.1 实验环境设置.....48
5.2 实验结果与分析.....50
5.2.1 服务执行需求与运营商收入的结果分析.....50
5.2.2 定价与服务购买量及运营商收入的结果分析.....51
5.2.3 最优服务器工作数量与最优运行速度.....54
5.2.4 与已有相关研究的对比.....58
5.3 本章小结.....60
第六章 总结与展望.....61
6.1 本文总结.....61
6.2 未来工作方向.....62
附录一 作者攻读硕士学位期间发表的科研成果.....63
附录二 本文相关性质的证明与公式计算.....64
A 关于需求分布性质的证明.....64
A-1 证明公式(2.10)中的负二项分布满足性质 1.....64
A-2 证明公式(2.10)中的负二项分布满足性质 2.....65
B 算法 1 中相关定理的证明.....65
B-1 定理 1 的证明.....65
B-2 定理 2 的证明.....67
参考文献.....68
致谢.....73

主要参数及含义对照表

参数符号	参数含义
t	云服务收费周期的长度
T	收费周期t内每个时隙的长度
m	收费周期t内实际购买云服务的用户数量
n	收费周期t内对云服务感兴趣的用户数(并不一定会发生购买行为)
ω	收费周期t内云服务的定价
θ	对用户费用返现的奖励因子
κ_i	表示i类型用户的收入损失因子
D_i	i类型用户服务请求的最大可允许延迟时隙
λ_1	收费周期t内对云服务感兴趣的用户的到达率
α,β	分别表示 Gamma 分布的两个参数
λ_2	用户服务请求到达多服务器集群的到达率
M	多服务器集群中服务器的数量
s	服务器的运行速度
ρ	表示服务器利用率，即服务器处于忙状态所占时间的百分比
$\overline{R}$	表示用户服务请求的响应时间
$\lambda_2^i[\tau']$	表示在第τ'个时隙内i类型用户服务请求的平均到达率
$\phi_i[\tau']$	表示在第τ'个时隙内i类型用户的服务请求被延迟的数量

插图和表格目录清单

图 1-1 研究内容整体架构图.....7
图 2-1 典型的三层架构模型.....10
图 2-2 t, T, D_i三者关系示意图.....17
图 2-3 多服务器系统模型.....18
图 2-4 M/M/M 队列模型.....20
图 3-1 子域分解算法.....33
图 3-2 求解最优服务定价与系统配置的 EALA 算法.....40
图 4-1 动态自适应闭环控制系统(DALCS)工作流程图.....43
图 4-2 DAA 算法流程图.....45
图 4-3 动态自适应(DAA)算法.....47
图 5-1 三种类型工作负载.....48
图 5-2 Ameren 实时电价.....49
图 5-3 运营商收入与服务执行需求之间的关系.....51
图 5-4 不同 λ_2 下的定价与服务购买量及运营商收入.....53
图 5-5 两种模式下利润与服务器数量之间的关系.....55
图 5-6 两种模式下利润与服务器运行速度的关系.....56
图 5-7 两种模式下的最优 M 与 s.....58
图 5-8 λ_2=16.9、M=17 时的最大利润对比图.....59
图 5-9 λ_2=12.55、M=18 时的最大利润对比图.....60
表 5-1 实验固定参数取值表.....49
表 5-2 验证执行需求与收入之间关系的参数表.....50
表 5-3 实验参数续表.....54

第一章 引言

1.1 研究背景与意义

云计算(cloud computing)是一种商业计算模型，它将计算任务分布在由大量计算机或服务器构成的资源池上，使用户能够按需获取计算服务、存储空间和信息服务[1]。作为一种联合各种计算资源和计算服务的高效方法，云计算已经变得越来越流行。通过集中管理互联网上的资源和服务，云计算提供基于 Internet 的服务托管方式，例如访问共享的硬件、软件、数据库、信息等其它所有按用户需求提供的资源[2]。云计算一般通过 pay-per-use 的价格模型，把信息技术转化成普通的商品和公共事业设备[3][4][5]。然而，云计算作为信息技术转化而成的普通商品永远都不会免费[6]，因此了解其商业运行模式及经济性至关重要。

在一个云计算的环境中，通常包括三层结构，即基础设施供应商、云服务运营商和用户[7]。基础设施供应商提供并维护最基础的硬件和软件设施。运营商通过从基础设施供应商处租赁所需软硬件资源来构建云集群环境向用户提供云服务。用户则向云服务运营商提交他们的服务请求，基于服务水平协议(SLA)中规定的服务可用性保证水平，从云服务运营商那里获得期望的运行结果，同时按照所提交的服务请求数量来缴纳费用并根据最终反馈的服务质量获取相应补偿[8]。

在整个三层架构中，云服务运营商租赁所需软硬件资源构建云集群，通过对用户收取一定的服务费用来执行用户提交的服务请求。正如前所述，作为一种商业运作模式，云服务运营商需要尽可能的使自己能获得最大的收益。然而，根据市场环境下的用户需求定律，若是云服务价格定的过高，相应的购买人数可能会降低，总利润可能减少；若是服务定价过低，虽然可能有较多购买人数，但是综合下来的利润也不一定会太高[9]。因此云服务运营商需要找到一个不但能获得较多服务购买量又能获得高额利润的最优云服务定价。此外，云服务运营商还需要对用户提交的服务请求有一个服务质量保证(QoS)。在调度用户的服务请求时不能毫无限制的延长用户服务请求的截止时间，否则会降低用户的服务满意度。不同的用户对其所提交的服务请求有不同的响应时间要求。例如，对于交互式的

服务请求，用户希望能够得到最快的响应，此类服务请求叫做非弹性服务请求；而对于非交互式的服务请求，用户自己有一个延迟时间容忍限度，此类服务请求叫做弹性服务请求。对于那些用户提交的能够容忍一定延迟时间的服务请求，云服务运营商有一个相对宽松的调度时间，所以在不超出用户服务请求所允许的延迟时间这一前提下，可以稍微往后安排调度。由于此类服务请求能带来较高的调度灵活性，所以对提交此类服务请求的用户应该给予一定的费用奖励。

关于云服务运营商的费用支出，除了上述所提的对提交弹性服务请求的用户做出奖励的支出外，另一个支出是维护其构建的云集群环境正常运行所需要的费用，主要是电费支出(跟能耗有关)和多服务器系统的租赁费用。租赁费用与服务器数量呈线性相关；而能耗除了与服务器数量呈线性相关外还与服务器运行速度的平方成正比例，它们都与服务器集群的硬件配置有关。除此之外，当用户提交的服务请求以一个低服务质量完成时，还应该给予用户一定的费用补偿，也就是说对运营商应该进行收费惩罚。上述费用支出对云服务运营商来说不可忽视。

云计算服务的价格关系到云服务运营商的收入。服务质量的高低会在很大程度上影响用户的服务满意度，后者关系到后续的服务购买量，而服务购买量又直接影响运营商的收入。此外，服务定价也直接影响着服务的购买量，合适的服务奖励措施也与用户的服务满意度紧密相关。因此需要寻求合适的多服务器系统配置(例如，工作服务器数量与运行速度)用以获得较高的服务质量、最优的云服务定价用以获得高服务购买量以及合适的服务奖励措施来获得较高的用户服务满意度。综上所述，在最大化用户服务满意度的前提下寻求最优云服务定价和多服务器系统配置，使得云服务运营商获取高额利润具有很高的研究价值。

1.2 国内外研究现状分析

云服务运营商的利润与很多因素有关，例如，云服务价格、市场需求、系统配置、用户的服务满意度等等。云服务运营商很自然的希望靠设置一个较高的价格来获得较高的利润，但是高服务收费的做法可能会降低用户关于云服务的满意度从而导致未来一段时间用户在该运营商处的需求降低[9]。因此选择一个合理的定价策略至关重要。随着云计算在商业市场上的发展成熟，作为一个商业计算

模型，不同提供云服务的运营商具有不同的定价策略。目前定价策略主要分为两种：固定定价策略及动态定价策略。

1.2.1 云服务中固定定价策略的研究

固定定价(Fixed pricing)策略是目前各大云服务运营商所采用的最普遍的定价方式，在所有的定价方式中占据着主导地位。固定定价意味着某个服务请求的价格固定且可提前预知，而且不会随着条件的改变而改变。在实际运作中，各大云服务运营商会根据自身实际运营情况进行细微调整，因此采用的固定定价方式会有所差异。在固定定价策略中有三类主要的定价方式：按使用量付费(Pay Per Use)，认购定价(Subscription based Pricing)及分级定价(Tiered Pricing)。

(1)按使用量付费(Pay Per Use)

在按使用量付费的定价模式中，云服务运营商根据用户使用的云计算服务的量，来向用户收取按使用量多少而缴纳的相应费用，使用量的衡量方式有如按小时计算或每 GB 流量计算或每小时 CPU 使用等。该定价方式允许用户根据自身对云服务的需求以及资金能力来选择或者组合购买所需云服务。基础设施即服务(IaaS)和平台即服务(PaaS)模式一般使用按使用量付费的固定定价方式。

基于使用量付费的定价方式不但直观且便于计算，同时用户也可以很明确地了解自己所使用的云服务以及支付了哪些方面的服务费用。对于云服务运营商来说，该收费方式在实际运营时具有很高的便捷性。目前，Amazon，Google、Microsoft 等大型云服务运营商的部分云服务采用这种定价方式向用户收费。

(2)认购定价(Subscription-based Pricing)

在该定价模式中，用户与云服务运营商签订有关云服务的使用合同，在某个收费周期内以固定的价格使用云服务。认购定价方式与按使用量付费方式比较类似，区别在于认购定价的收费单位是单位时间而不是单位使用量。在软件即服务(SaaS)模式上认购定价是使用最广泛的定价模式。目前，Salesforce，Adobe，XDrive 和 XCalibre 等公司都有采用认购定价方式。

(3)分级定价(Tiered pricing)

分级定价，顾名思义是指云服务运营商将云服务分为不同的等级，不同等级的档次不同收费也不同。虽然等级不同，但是每个等级都有自己完整的计算规格，

例如存储大小，CPU 速度，带宽，虚拟机内存大小等。在向用户提供云服务的过程中，都需要遵守服务等级协议(SLA)。用户根据自己的需求选择不同的等级进行购买使用，通常每个等级按照单位时间制定价格。

阿里云、Google、Amazon 和 Microsoft 等大型云服务运营商采用了该种收费方式，向用户提供不同等级的云服务。例如阿里云提供不同种等级的虚拟服务器实例，每个等级所需的费用随着配置的内存、虚拟内核速度、通信带宽等性能的增高而逐渐增高。

固定定价策略目前是在现实世界和相关文献研究中使用最多的也是最广泛的定价机制[2][10][11]。文献[11]中，Ghamkhari 和 Mohsenian-Rad 采用了一种叫做 flat-rate 的定价策略，对于所有的请求设置一个固定的价格；但是文献[12]中，Odlyzko 则论证 flat-rate 会导致资源浪费，同时也不能适应服务差异化。另一种固定定价策略是基于使用量(usage-based)，例如服务的价格分别与服务时间以及服务的执行需求(以待执行的指令条数来衡量)成比例[10][2]。这种定价策略(usage-based)可以使得资源更有效的利用[13][14]。尽管固定定价策略有不同的实现方式，但是都既不能满足用户的动态需求也不能反映市场的供需状况。因此需要一个能解决动态用户需求的定价策略，即动态定价策略。

1.2.2 云服务中动态定价策略的研究

动态定价(Dynamic Pricing)是指云服务运营商根据实际运营中用户对云服务的认可、服务本身的价值以及根据市场供需状况动态地调整云服务的价格。对于动态定价策略，云服务运营商延迟对其所提供的云服务进行定价直到用户的需求状况明确，然后据此制定价格提供云服务并向用户收取费用。因此，该定价策略允许同一类型的商品能够根据时间段、市场供需情况、用户的感知价值等的改变而动态地调整云服务价格。

动态定价策略是作为一个有吸引力的为了更好解决不可预测的用户需求而出现的[15]。Macias 和 Guitart [16] 提出了一个用遗传算法以迭代方式获得最优定价的策略；Amazon EC2[17][18]提出了一个“spot pricing”的策略，其中“spot price”对于某一个虚拟服务实例动态地调整价格以适应供给和需求的变化。然而用户不太喜欢随意的价格变动，尤其是当他们感知到价格变动是“不公平”（比

如，某个时间段之后由于需求的改变导致价格变的比之前更高)的时候[19][20]。文献[26]中，Yeo 等分析并比较了云服务运营商现采用的服务定价策略，文中所得结论是固定定价已经无法满足用户多样化的需求，动态定价策略作为一种能根据外界条件的改变而动态改变服务价格的方式，既能满足用户的动态需求又能使得云服务运营商获得高额收益。

虽然动态定价策略是为了更好的解决用户需求的不可预测性，但是由不确定性所导致的需求建模及求解的复杂性限制了该方案在现实世界中的应用。本文基于经济学理论中的用户感知价值理论，构建了一个能反映用户需求定律的动态定价模型。此模型不但解决了不同定价下用户需求的不可预测性，又解决了基于此模型所构建的利润最大化模型的求解复杂性问题。

1.2.3 关于云服务运营商利润最大化的研究

影响云服务运营商利润的因素主要有两个：云服务定价策略以及用户的服务满意度。定价策略是影响云服务运营商利润的第一个也是最重要的一个因素。本章前两节讨论了关于固定定价策略以及动态定价策略的研究现状。选择一个合适的定价策略是使得云服务运营商利润最大化的最低要求。文献[26]中，Yeo 等将时间、需求以及资源对云服务运营商利润的影响进行了深入分析，为了解决用户对云服务的不同需求及要求并最大化云服务运营商利润，该文献提出了一种基于高峰期/低谷期自动计量的定价策略。文献[27]中，为了研究资源定价(Resource Pricing)、捆绑定价(Bundled Pricing)及差异化定价(Differentiated Pricing)三种动态定价策略下云服务运营商的利润以及不同用户采用不同定价的公平性，Joe 等人建立了一个数学模型来评估三种策略之间的不同。文献[28]主要针对动态定价策略，根据用户对云服务的不同需求情况，构建了一个资源与价格相映射的模型，通过该模型有效分配和利用资源并最大化云服务运营商利润。

第二个影响云服务运营商利润的因素是用户的服务满意度。用户服务满意度是由服务的质量以及服务收费共同决定。为了提高用户的满意度的层级，在用户和云服务运营商之间有一个服务等级协议(Service Level Agreement, SLA)。SLA 采用了一种价格补偿机制，对于低服务质量的用户将会进行一定的价格补偿。这种机制是为了保证服务质量以及用户满意度，因此可以吸引更多的用户。已有的

研究工作中采用了有很多不同方式的 SLA。文献[2]中采用了一个价格补偿机制充当 SLA，当服务质量低下时降低收费。文献[11]中，Ghamkhari 和 Mohsenian-Rad 采用了一种含有两个阶层的阶梯式收费方式来保证用户的服务满意度，若一个服务请求在它的截止时间之前被处理则正常收费，当超时之后则由云服务运营商为之买单作为超时惩罚。在文献[2][10][29]中，云服务收费随着服务请求等待时间的增长而持续降低直到免费为止。

获取最大利润作为云服务运营商的最重要目标，已经为之有很多工作。文献[21][22][23][24]中大部分的工作都是集中在通过降低能耗费用来降低云服务运营商的支出费用，从而提高利润。主要采用如空闲服务器关闭策略以及动态 CPU 时钟频率缩减等技术用来降低能耗费用。然而仅仅降低能耗费用不能保证利润最大化[2]。因此，很多工作研究的目标转向最小化耗费和最大化收入之间的平衡以使得利润最大化。文献[11][25][30]使用不同的策略周期性的调整开启服务器的数量，并建立了不同的利润最大化模型来获得需要开启的服务器数量，以此来达到收入和耗费之间的平衡。

尽管通过降低能耗的措施可以提高云服务运营商的利润，然而这些利润最大化的定价策略并没有把用户的行为因素考虑进去，没有解决用户需求具有动态性的问题，因为不同时段、不同定价下用户对服务的需求也不同。也就是说上述文献中所做的定价仅仅是理论上的一个最优定价。该定价是否符合市场条件下的用户需求分布，是否符合用户的需求定律，并不是简单的做个某两个因素之间的权衡就是最优的。此外，这些研究工作集中在云计算集群部分进行研究，然而云计算作为一种商业应用的计算模式，用户的因素必须考虑进去，例如用户对不同紧急程度服务请求的不同执行需求，服务请求超时补偿，对弹性服务请求的费用返现奖励，最优的系统运行配置，服务等级协议(SLA)，用户的服务满意度等等。

1.3 本文的研究目标和内容

本文主要研究的是在云计算环境中，基于用户的感知价值考虑市场条件下用户的需求分布，建立满足用户需求定律的云服务定价模型，在保证用户服务满意度的前提下，获取最优的云服务定价和多服务器系统运行配置使得云服务运营商的利润最大化。除此之外，本文构建了一个动态自适应闭环控制系统模型，使得

本文提出的研究方法能动态适应市场条件的改变，例如用户随着市场条件下某种服务价格的改变而改变对该服务的感知价值，电费的改变，租赁费的改变等。在上述目标的研究过程中，本文提出的价格模型考虑了如下因素：服务请求的数量，服务请求的紧急程度，用户的服务满意度，低服务质量时的惩罚，应用环境的工作负载，多处理器集群的配置，服务等级协议(SLA)，服务质量(QoS)，对弹性服务请求的费用返现奖励，多服务器集群租赁费用，能耗费用，市场环境中用户的动态需求分布。本文研究内容的整体架构图如图 1-1 所示。

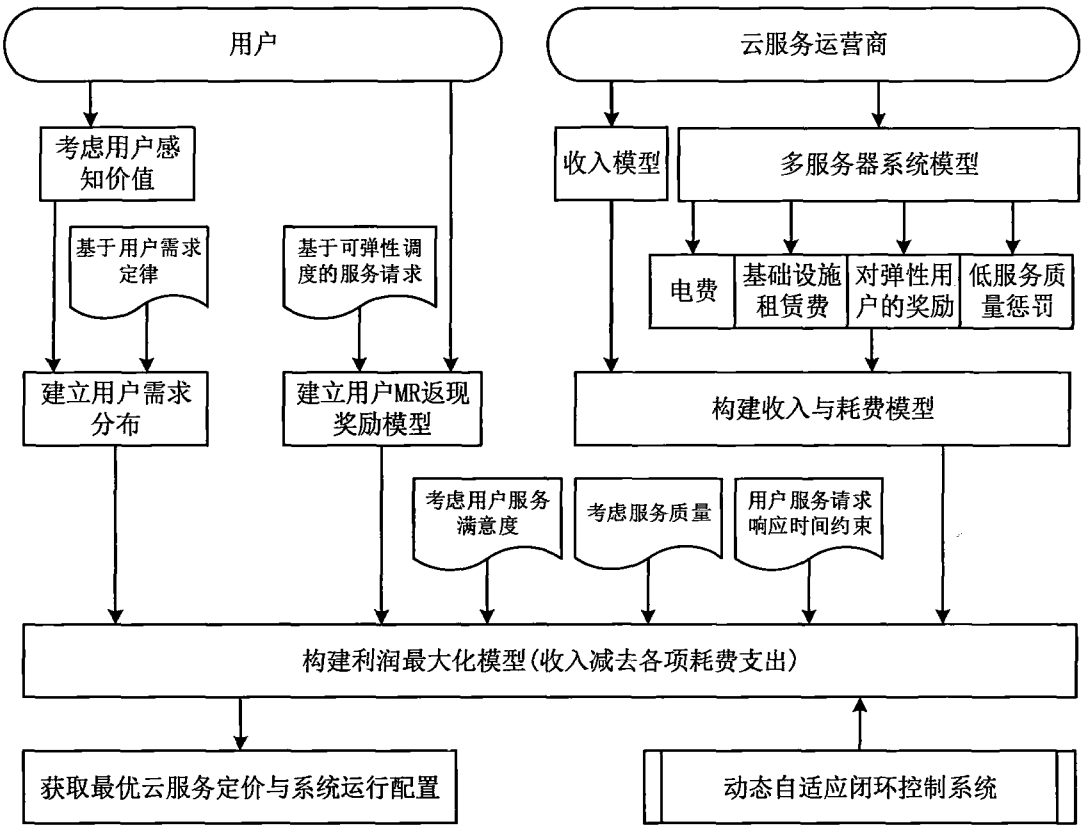


图 1-1 研究内容整体架构图

如上图所示，本文综合考虑了用户的行为因素和云服务运营商的收入与各项耗费支出。为了解决用户的动态需求，本文考虑用户的感知价值，建立了一个能反映用户需求分布的定价模型，即图中所示的用户需求分布。此外，为了获得较高的用户服务满意度，本文建立了基于服务请求延迟敏感度的 MR 奖励模型。关于云服务运营商，本文为其建立了收入模型以及多服务器系统下的耗费模型。然后根据所建立的用户与运营商的模型，构建了云服务运营商的利润最大化模型，

该模型综合考虑了用户的服务满意度，服务质量以及 SLA 中关于响应时间的约束。最后应用本文提出的动态自适应系统模型，使得系统能动态调整运行参数。

1.4 本文的创新点

本文针对云服务运营商的利润最大化问题进行研究，具有以下创新点：

- 通过对云服务链中涉及的不同利益主体(主要是用户和云服务运营商)的特征及定价策略进行分析，提出以“用户”和“用户感知价值导向”为云服务价值感知主体的动态云服务定价策略设计思想，并以此作为构建本文云服务定价模型的切入点。
- 在云服务定价模型上，考虑市场环境下云服务定价与用户需求分布之间的动态关系，据此提出了一个基于用户感知价值的符合用户需求定律的云服务定价模型。该定价模型能解决用户需求的动态性。
- 按照用户对其服务请求响应时间的不同需求，以及不同服务请求对延迟调度的不同敏感性，根据实际情况将用户的服务请求按照紧急程度分成若干类，用户可以按照自己的需求自行设定服务请求的紧急程度，即是否允许调度延迟。基于此本文提出了一个费用返现奖励(Monetary Reward, MR)模型，用户可以通过设定其服务请求允许的延迟时间长度来从云服务运营商处获得相应的返现奖励。这种方式下，由用户自主设定其服务请求是否允许延迟，奖励多少完全由用户的设定所决定，此举意在获得较高的用户服务满意度。
- 为了提高用户的服务满意度，除了上述的 MR 策略，我们的研究还考虑了服务等级协议(SLA)。本文基于用户服务请求的响应时间构建了一个低服务质量时的收费惩罚模型。对运营商来说低服务质量时需要受到收费惩罚，对用户来说则需要一定的补偿。对运营商的惩罚费用按照用户服务请求的响应时间呈现阶梯式。
- 为了使本文提出的研究方法能动态适应外界条件的改变，例如用户对云服务的感知价值、电价等的改变，本文构建了一个动态自适应闭环控制系统模型。该系统模型具有根据历史数据进行自主学习能力，使得本文的算法能动态自适应外部条件的改变。

1.5 本文的组织结构

本文的整体结构如下：

第一章介绍了本文的研究背景，重点介绍了国内外关于云服务定价策略与利润最大化的研究现状。分析了如今云服务定价相关研究的不足之处，最后简单概括了本文的研究内容与创新点。

第二章主要介绍本文研究过程中涉及到的系统架构和模型。包括系统中两类主要角色用户和云服务运营商的相关模型。主要包括，用户的需求分布模型，多服务器系统模型，云服务运营商的收入模型、电费模型、租赁费用模型、对提交弹性服务请求用户的返现奖励模型以及低服务质量时的收费惩罚模型。

第三章设计了求解最优云服务定价与最优系统配置的算法。本部分主要包括求解问题的定义及相关约束条件的说明，求解最优奖励因子的理论分析与算法设计，求解最优定价的算法设计、求解最优系统配置的算法设计。

第四章对本文提出的动态自适应闭环控制系统模型进行了介绍。具体包括动态自适应闭环控制系统模型的流程介绍、理论分析以及算法设计。

第五章为本文研究内容的仿真实验以及结果分析。包括，对本文所提出的云服务定价模型的实验验证与结果分析，最优系统配置参数的结果分析以及与已有相关研究工作的对比与分析。

第六章总结本文所做的工作并提出了未来的研究方向。

第二章 系统架构和模型

本章主要介绍云计算环境中典型的三层架构模型，以及基于该架构涉及到的两个主要角色：用户与云服务运营商的相应模型。后文将基于这些模型对利润最大化问题下的最优云服务定价与多服务器系统配置进行研究。

2.1 典型的云计算架构模型

一个典型的云计算架构模型通常有三层，依据这三层分为三类角色：基础设施供应商，云服务运营商以及用户[7]。这种三层架构模型在已经存在的研究工作中使用很普遍[2][10][31]。

基础设施供应商提供并维护最底层的硬件和软件设施；云服务运营商从基础设施供应商处租赁所需软硬件资源构建云计算集群环境并向用户提供云服务；用户则向云服务运营商提交他们的服务请求，即任务，并基于服务等级协议(SLA)中规定的服务可用性保证水平，从云服务运营商那里获得期望的运行结果，同时按照所提交的服务请求数量来缴纳相应的费用。

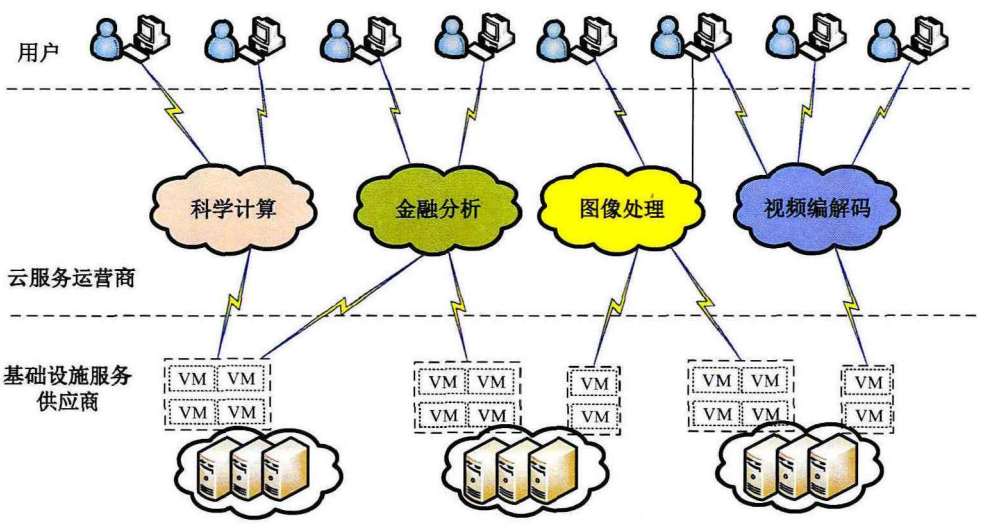


图 2-1 典型的三层架构模型

如图2-1所示，云服务运营商租赁相关的软硬件资源构建自己的云集群环境，并向用户提供如科学计算、金融分析、图像处理及视频编码等云服务。

2.2 用户模型

为了最大化利润，云服务运营商需要知道市场中所有用户的总需求函数。当云服务运营商设定了服务的价格之后，不同的用户对此价格具有不同的反应。与传统市场中的商品类似，云计算作为一种商业模式，云服务可以被认为是一种特殊的商品，仍然遵循市场条件下的用户需求定律。也就是说，云服务的价格仍然受供需关系影响。基于用户感知价值理论[32]，本文提出了以用户感知价值为导向的定价策略。本部分，我们主要介绍用户感知价值以及用户需求分布模型。

2.2.1 用户感知价值

在传统的市场环境中，考虑一个具有一系列不同商品且每件商品定价都很明确的商店。以广告促销活动举例，在正常的销售期内，商店通常为一系列商品提供折扣范围而并不明确指出单个价格。大部分情况下，商店使用代表性的商品样本来进行促销，顾客提前并不知道每件商品的具体价格[9]；顾客到达商店的到达率往往是他们定期购买模式的一种反应而不是对个别价格的反应[9][32]。例如，在到达商店之前客户并不知道也不需要知道商品的确切定价，他只需要知道商店里有他感兴趣的商品即可，至于到店参观后是否会发生购买行为，这与其对商品的感知价值有关系。因此，我们假设客户在到访商店之前不知道每件商品的确切价格。此外，文献[9]指出商品标价的变化对在正常销售期间访问商店的顾客总数没有影响。另一方面，并不是所有到达商店的顾客都有购买某个特定商品的意愿，也就是说购买商品的人数不大于到达商店的人数。

用户的感知价值是指顾客所能感知到的利益与其在获取产品或服务中所付出的成本进行权衡后对产品或服务效用做出的整体评价[32][33]。在市场环境中，只有那些对商品的感知价值超过定价的顾客才会发生购买行为[34]。通俗的讲，用户购买某件商品之前会对该商品有个预期的价格，该预期价格表示了用户对此商品所能接受的价格的上限。一旦商品的定价低于该预期价格，用户一般会发生购买行为，也就是说用户的感知价值超过商品定价。

与传统市场环境中的商品类似，云服务可以被看作是一种特殊的商品。如果将提供云服务的运营商视为一个大商店的话，那么不同价格不同档次的云服务就

是该商店里的各种商品。一旦将定价明确的云服务推向市场之后，其仍然受用户感知价值的影响，该服务的购买量也受市场条件下用户需求分布的影响。

2.2.2 用户需求分布

由于云服务是一种特殊的商品，为了方便研究满足市场需求定律的动态用户需求分布，本文把提供不同云服务的运营商视为拥有不同商品的商店；将某个定价下对云服务感兴趣的用户视为到达商店的客户，当然客户并不一定会购买服务，与客户的感知价值有关；将实际购买云服务的用户视为到达商店并愿意以当前商品定价购买商品的客户。

假定参数ω表示单个云服务在每个收费周期内的定价，由于本文采用的是动态定价策略，所以不同收费周期的服务定价ω可能有所不同。假设用m表示当云服务价格设定为ω时实际的购买人数，用$E[m|\omega]$表示定价为ω时购买服务的人数的期望。文献[35][36]中，采用实际购买人数的期望$E[m|\omega]$来衡量用户的需求分布。与此不同的是，本文采用基于实际购买人数总需求的概率分布而不是期望来衡量用户的需求分布。这样可以比较真实的反映市场环境中用户的实际需求情况。

用t表示云服务正常销售期的长度，即一个收费周期的时长。假设$P_{\omega}[m]$表示在收费周期t内，云服务定价为ω时有m个用户购买的概率，显然$P_{\omega}[m]$是一个符合负二项分布的概率质量函数。一个合适的需求函数应该能正确反映市场环境中的用户需求分布，符合用户需求定律。因此概率质量函数$P_{\omega}[m]$若想作为一个合适的需求函数，必须满足以下两个性质。

【性质 1】用户的需求与价格呈反方向变动关系。

第一个性质就是用户对商品的需求数量与价格成反向变动关系，价格越高购买的用户数越少。随着云服务价格的增高，用户的需求应该减少。例如从直观上理解，对于定价分别为ω和$\omega+\Delta,(\Delta>0)$的两种商品，n个人中只有一个人购买定价为ω的商品的概率比只有一个人购买定价为$\omega+\Delta$的商品的概率要低。换句话说购买定价为ω的商品的实际人数应该多于一个，用户更倾向于同等质量下的低价商品。因此，参考文献[9]我们有如下的需求分布性质：

$$\sum_{m=0}^{c} P_{\omega}[m] \leqslant \sum_{m=0}^{c} P_{\omega+\Delta}[m], \tag{2.1}$$

其中，$c = 0,1,2,\cdots,\infty$；$\Delta > 0$；当且仅当$c = \infty$时，上述等式成立。

上述条件等价于：

$$\lim_{\Delta \to \infty} \frac{\sum_{m=0}^{c} P_{\omega+\Delta}[m] - \sum_{m=0}^{c} P_{\omega}[m]}{\Delta} = \sum_{m=0}^{c} P_{\omega}'[m] \geq 0, \tag{2.2}$$

公式(2.2)对于任意$c = 0,1,2,\cdots,\infty$成立，其中$P_{\omega}'[m]$表示需求函数$P_{\omega}[m]$在ω处的一阶偏导数。当$c = \infty$时，对任意关于m的概率质量函数上式等式部分都成立。在大部分研究定价策略的文献中，一般都认为市场需求会随着定价ω的增高而降低[9][35]，性质 1 也符合这一点。然而并不是任何满足公式(2.2)的概率分布函数都能作为需求函数，例如：

$$P_{\omega}[m] = \frac{\omega^{m} \exp(-\omega)}{m!}, \tag{2.3}$$

其中，$m = 0,1,2,\cdots,\infty$；这是因为需求随着价格ω的增高而增高。所以我们需要加上额外的限制条件。

【性质 2】当价格趋向于无穷时，用户需求的期望趋向于 0。

第二个需要满足的性质是随着云服务价格ω的增大并趋向于无穷，购买云服务的用户数量趋向于 0；此时，云服务运营商出售云服务商品失败，即收入趋向于 0。这也比较符合实际的情况，过高的价格用户无法负担导致需求降低。参考文献[36][37]中的相关理论，还需要对需求分布函数加上如下的条件限制：

$$\lim_{\omega \to \infty} \omega \cdot E[m \mid \omega] = 0, \tag{2.4}$$

其中，$E[m|\omega]$表示定价为ω时购买云服务用户数量的期望。例如，下面的式子不能作为需求分布函数：

$$P_{\omega}[m] = \begin{cases} \dfrac{\omega}{1+\omega}, m = 0 \\ \dfrac{0.5}{1+\omega}, m \geq 1 \end{cases}, \tag{2.5}$$

尽管公式(2.5)中的式子满足由公式(2.2)所表示的限制条件，但是其期望$E[m|\omega]=2/(1+\omega)$，并不满足由公式(2.4)表示的条件(因为此时期望不等于0)，所以不能作为需求函数。一般地，把满足上述两条限制性质的需求分布函数称为常规需求分布[35]。

在收费周期t内，当运营商将云服务价格设定为ω时，用n表示对该定价下的云服务感兴趣的用户数，其中$n=0,1,2,\cdots,\infty$，为离散随机变量。由于到访商店的顾客总数与商品的定价相互独立，因此有理由假定随机变量n与系统中的其它参数变量也相互独立[9][35][36]。假设用X_j表示第j位用户愿意购买云服务的最高可接受价格，其中$0\le X_j<\infty$是一个连续随机变量。与文献[36]中的定价模型类似，$(X_1,X_2,\cdots,X_n)$是独立同分布的连续随机变量，其分布的概率密度函数设为$f(x)$，通常是已知的或可以先验地估计。在ω处评估的x的累计分布函数由$F(\omega)$表示。当云服务定价为ω时，$F(\omega)$是一个非递减函数，并且满足常规概率分布的性质：$0\le F(\omega)\le 1$。由于ω必须取有限的非负值，所以在ω处的累计分布函数具有以下特征[38]:

$$\lim_{\omega\to\infty}F(\omega)=1, \tag{2.6}$$

把收费周期t内云服务实际购买用户数m定义为：n个对云服务感兴趣的用户中愿意以当前定价购买服务的用户数量，即所有感知价值$X_j\ge\omega$的顾客人数。则实际购买服务的用户数m可以表示为一个服从二项分布的随机变量，表示如下：

$$P_\omega[m\,|\,n]=\binom{n}{m}\times[1-F(\omega)]^m\times[F(\omega)]^{(n-m)}, \tag{2.7}$$

其中，$m=0,1,2,\cdots,n$。

参考文献[35]中的定价模型，本文假定收费周期t内所有对该定价下的云服务感兴趣的用户数n服从参数为λ_1的泊松分布：

$$P(n\,|\,\lambda_1)=\frac{(\lambda_1 t)^n e^{-\lambda_1 t}}{n!}, \tag{2.8}$$

其中，$n=0,1,2,\cdots,\infty$，λ_1表示单位时间内对云服务感兴趣的用户的到达率。值得注意的是，公式(2.8)中对云服务感兴趣的用户总数n与云服务的定价ω无关，而公式(2.7)中实际购买云服务的用户数量m则是关于定价ω的函数。

通常情况下，收费周期t内不同时间段的用户到达率λ_1具有较大的波动性，为了表示这种到达率的非均匀性，用参数为(α,β)的 Gamma 分布来表示：

$$g(\lambda_1)=\frac{1}{\Gamma(\alpha)\beta^{\alpha}}\lambda_1^{(\alpha-1)}e^{-\lambda_1/\beta},\ 0\le\lambda_1\le\infty, \tag{2.9}$$

其中，$E(\lambda_1)=\alpha\beta$，$\mathrm{Var}(\lambda_1)=\alpha\beta^2$；$\Gamma(\alpha)$表示 Gamma 函数，与$\alpha$的阶乘相关。之所以选择 Gamma 分布作为$\lambda_1$的概率分布，原因如下：首先，它与公式(2.8)中的泊松分布共轭；其次，达率λ_1是一个非负取值的随机变量；最后，选择 Gamma 分布可以涵盖较多的λ_1的变动情况，因为它可以表示λ_1的概率密度函数的各种形状，例如，当(α,β)取固定值时上述的 Gamma 分布将转化为相应地χ^2分布或者指数分布。因此，本文选择 Gamma 分布来反映到达率的波动性。

综合公式(2.7)(2.8)(2.9)，当收费周期t内的云服务定价设为ω时，实际购买云服务的用户数量m的概率可以表示如下：

$$\begin{aligned}P_{\omega}[m]&=\int_{\lambda_1=0}^{\infty}\sum_{n=0}^{\infty}P(m\mid n)P(n\mid\lambda_1)g(\lambda_1)d\lambda_1\\&=\binom{m+\alpha-1}{m}\left[\frac{\beta t[1-F(\omega)]}{1+\beta t[1-F(\omega)]}\right]^{m}\left[\frac{1}{1+\beta t[1-F(\omega)]}\right]^{\alpha},\end{aligned} \tag{2.10}$$

其中，$m=0,1,2,\cdots,\infty$。可以看出，收费周期t内用户的实际购买需求服从一种负二项分布，且定价为ω时其期望为：

$$E_{\omega}[m]=\alpha\beta t(1-F(\omega)). \tag{2.11}$$

公式(2.6)给出了任何累积分布函数$F(\omega)$的所具有的限制性质，根据该性质可以证明公式(2.10)中关于m的负二项分布满足由公式(2.2)所表示的限制性质 1；同时可以证明，上述负二项分布也满足由公式(2.4)所表示的限制性质 2。限于篇幅，相关证明分别参见附录二[A-1]及[A-2]。

综上，公式(2.10)满足性质 1 与性质 2，符合要成为一个能衡量用户需求分布的模型所必须满足的限制条件。所以本文建立的如公式(2.10)所示的基于用户感知价值的需求分布模型，能作为市场环境条件下的用户需求分布模型，能满足用户需求分布的动态性。

2.2.3 用户 MR 返现奖励模型

本节主要介绍基于用户服务请求允许延迟时隙长度的返现奖励(Monetary Reward, MR)模型。具有不同工作负载的用户对延迟其工作负载具有不同的敏感性。例如，基于 MapReduce 的批处理工作负载通常可以容忍较长的延迟，而不会影响用户的其它正常业务；与此相反，用户交互式的工作负载，如 Web 搜索，通常不能容许太长时间的延迟，用户希望尽快的得到搜索结果。本文将使用云服务的用户分为I种类型，每个类型的用户对于工作负载的延迟具有不同的敏感性。更具体地说，本文使用收入损失因子κ_i $(\kappa_i \in \mathbb{R}_0^+)$来量化$i$类型用户对推迟其工作负载的敏感性：对延迟越敏感，$\kappa_i$越大。对于那些运行不容许有延迟容差的交互式应用程序的用户，我们设置$\kappa_i \to \infty$，并将这些用户称为非弹性用户。剩下类型的用户被称为弹性用户，该类用户的工作负载可接受在某种程度上的调度延迟。

对i类型用户来说，所谓的收入损失是指：由于其工作负载对资源需求的紧急程度要求不同，最终获得的奖励与预期奖励费用之间的差异。对延迟越敏感的服务请求，云服务运营商调度的压力越大(例如非常紧急的任务必须立即安排调度尽管此时服务器可能是满负载状态)。因此云服务运营商会从预期的奖励中扣除一部分调度费用，用户最终从云服务运营商处获得的奖励会有所降低。

假设用D_i表示i类型用户的工作负载可容许的最大调度延迟时隙长度，同样假定该类用户的潜在收入损失在D_i上非递减且是凸的。事实上，用户的工作负载通常在最大延迟时隙D_i到来之前得到处理，因此D_i可以被视为是i类型用户所能容许的最大延迟时隙。如果用T表示收费周期t内每个时隙的长度，则t, T, D_i三者的关系如图 2-2 所示。值得注意的是，延迟时隙D_i可能跨越几个时隙长度。

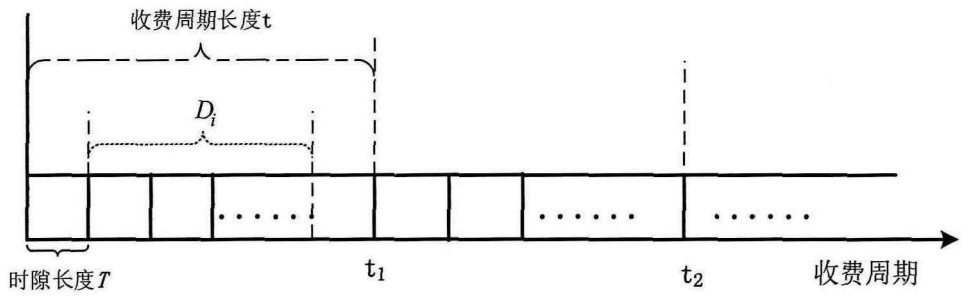


图 2-2 t,T,D_i三者关系示意图

由于用户的服务请求具有不同程度的敏感性，从而对云服务运营商来说具有不同的调度压力，所以运营商对用户的奖励根据服务请求敏感性将会适当的减少。参考文献[30]中的收入损失模型，用L_i表示来衡量i类型用户的收入损失，则有：

$$L_i = \kappa_i D_i \ , \tag{2.12}$$

用$\hbar_i$表示i类型用户由于其工作负载的灵活性所获得的奖励，则$\hbar_i$表示为：

$$\hbar_i = \theta \log(1 + D_i) \ , \tag{2.13}$$

其中，θ表示奖励因子，通常非负。此外由用户所设定的延迟时间上限应该满足：

$$0 \le D_i \le D_{max} \ , \tag{2.14}$$

其中，$D_{max} \in \mathbb{Z}_0^+$表示系统中最大的延迟时隙长度，是一个固定的正整数。

假定每一个用户都比较理性，也就是说每一个用户提交任务时都在延迟时隙上限D_i上做决定。通过设定合适的D_i来使得自己获得最大的返现奖励，同时也能容许由自己所设定的调度延迟时隙长度。因此，用户的最优化问题可以表示如下：

$$\begin{aligned} &\underset{D_i}{\text{maximize}} : \hbar_i - L_i \\ &\text{subject to} : 0 \le D_i \le D_{max} \end{aligned} \ , \tag{2.15}$$

其中，$\hbar_i$和L_i分别由公式(2.13)和(2.12)表示。尽管D_i应该设置为整型数值，但是本文中我们将其拓宽到连续变量类型以此来简化问题(2.15)的分析。因此，最优化问题(2.15)是凸问题，其解为:

$$D_i = \max(\min(\frac{\theta}{\kappa_i} - 1, D_{max}), 0) \ . \tag{2.16}$$

本文将上述基于用户服务请求的延迟敏感程性而给予用户相应返现奖励的方法叫做用户的 MR(Monetary Reward)奖励返现模型。

2.3 云服务运营商模型

云服务运营商构建好云计算环境，通过执行用户提交的服务请求向用户收取费用。另一方面，云服务运营商为了维持整个云计算平台的正常运转，必须支付相应的耗费费用，主要包括电费，低服务质量时的惩罚费用，对提交弹性服务请求的用户进行返现奖励，租赁费。本部分首先对云计算平台中的多服务器系统进行建模；然后构建云服务运营商的收入模型、耗费模型、对弹性用户的奖励模型以及低服务质量的惩罚模型。

云服务运营商通过使用多服务器系统来执行用户的云服务请求。通过相关研究文献不难发现，云平台上的多服务器系统其构造细节具有很大的灵活性。例如，刀片服务器(blade servers)和刀片中心(blade centers)，其中每一个服务器是一个服务器刀片[39]，也就是说每一块“刀片”实际上就是一块系统主板；传统的服务器集群，其中每个服务器就是一个普通的处理器[40][41][42]；多核服务器处理器，其中每个核代表一个服务器[43]。简单起见，我们将这些刀片/处理器/核心统一称作服务器，如图 2-3 所示[51]。

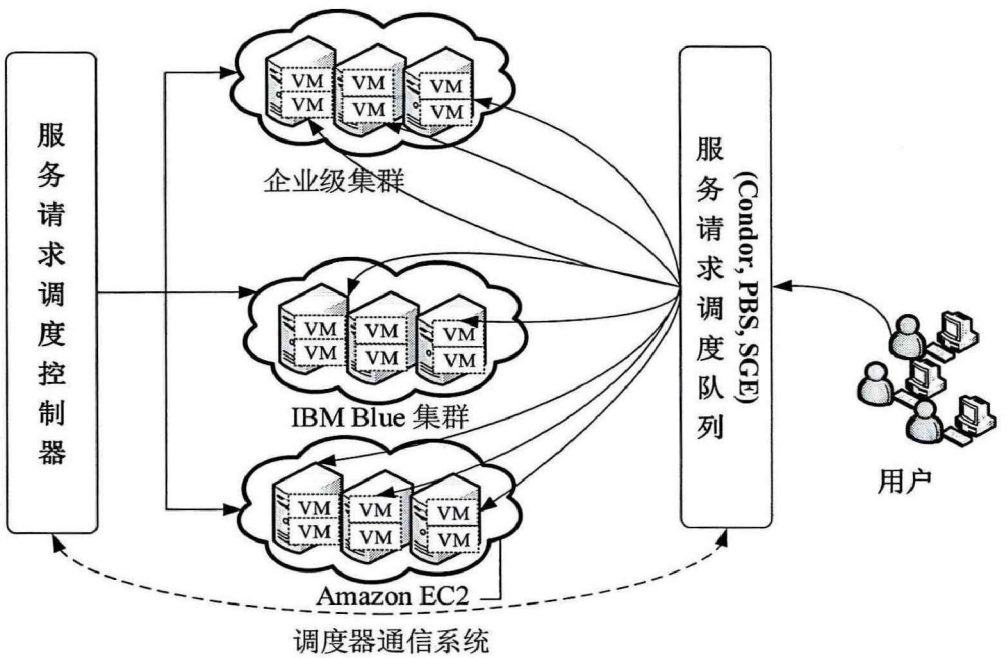


图 2-3 多服务器系统模型

如图 2-3 所示的多服务器系统模型，在实际的云计算平台(如 Amazon EC2，IBM Blue 云和企业级集群)中，云处理器如 Eucalyptus，OpenNebula 和 Nimbus 都有许多工作节点。云以虚拟机的形式为用户提供资源。用户将服务请求提交到采用 SGE，PBS 或 Condor 等任务排队系统的云端。所有服务请求都由服务请求调度控制器进行调度，并以集中的方式分配给不同的虚拟机(VM)进行处理。例如，Condor 是用于计算 CPU 密集型服务请求的专用工作负载管理系统，它提供了一套完整的服务请求排队机制、调度策略、优先级策略、资源监控和资源管理机制。用户向 Condor 提交服务请求，Condor 将它们放入队列中，根据策略选择何时何地运行它们[52][53]。因此，将云服务平台抽象为具有服务请求队列的多服务器模型是合理的，并且该模型在现有文献中被广泛采用[2][11][44][54]。不失一般性，本文也采用基于服务请求队列的多服务器系统模型。

2.3.1 多服务器模型

假设如图 2-3 所示的多服务器系统S具有M个同构服务器。本文中，多服务器系统S被看作是一个 M/M/M 队列系统，即到达服务器的服务请求遵循泊松分布。文献[44]考虑了一个叫做 M/G/k 的队列系统。然而对于这种队列系统，大部分的性能指标不能提前确定，例如等待时间。与此相反，M/M/M 排队模型系统是唯一适合对新服务请求等待时间的概率密度进行分析的模型，其中第一个 M 表示无记忆的泊松过程，第二个 M 表示用户服务请求的服务时间服从指数概率分布，第三个 M 则表示多服务器系统中服务器的数量。

对于多服务器系统 S 中的 M/M/M 队列系统，假设其具有服务请求到达率服从参数为λ_2的泊松流。每个服务请求的到达时间是独立同分布的指数随机变量，且平均到达率为$1/\lambda_2$。当所有的服务器都处于忙的状态时，新到达的服务请求不能被立即处理，为了保存到达的用户服务请求，多服务器系统维持一个具有无限容量的队列，后续到达的服务请求在此队列中等待调度直到可以被任何处理器处理为止。这里，本文采用先来先服务(FCFS)的调度策略。用户服务请求的执行需求由待执行的指令数来衡量，由一个服从独立同分布的指数随机变量r表示，用$\bar{r}$表示平均执行需求。本文假设 M 个服务器具有相同的执行速度，用s表示，由

单位时间内执行的指令数衡量。如果用x_1表示用户服务请求的执行时间，则x_1是一个指数随机变量且有$x_1 = r/s$，其中服务请求的平均执行时间 $\overline{x_1} = \bar{r}/s$。假设用$\mu$表示平均服务率，即单位时间内平均完成的服务请求数量。由于$\overline{x_1}$表示服务器请求的平均执行时间，因此，$\mu = 1/x_1 = s/\bar{r}$。

本文采用的 M/M/M 队列模型如图 2-4 所示，当到达的服务请求无法立即被处理，它们首先会被放入该队列中，直到可以被分配到任何一个服务器处理为止。

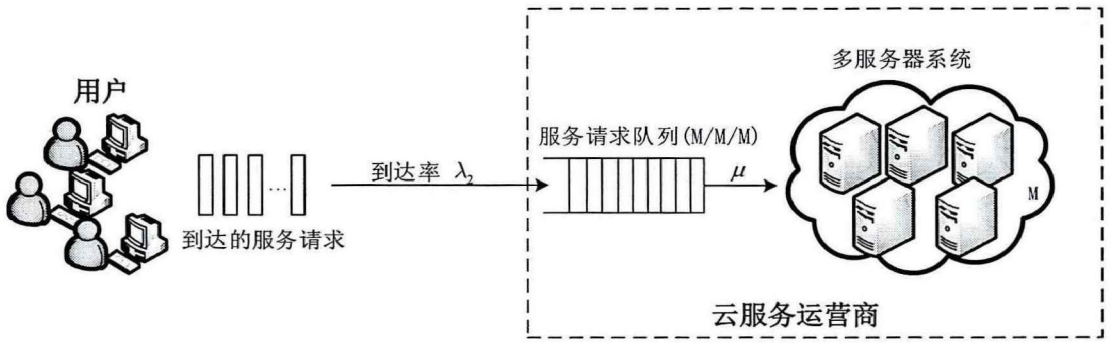


图 2-4 M/M/M 队列模型

假设用ρ表示服务器利用率，即服务器处于忙状态所占时间的百分比。则服务器利用率ρ可以表示如下：

$$\rho = \lambda_2/(M\mu) = \lambda_2/M \cdot \bar{r}/s \ . \tag{2.17}$$

用p_k表示 M/M/M 队列系统中有k个服务请求(正在被处理或处于等待状态)的概率，参考文献[45]可得：

$$p_k = \begin{cases} p_0 \dfrac{(M\rho)^k}{k!}, k \le M \\ p_0 \dfrac{M^M \rho^k}{M!}, k \ge M \end{cases} , \tag{2.18}$$

其中，p_0表示队列中没有服务请求的概率，表示如下：

$$p_0 = \left(\sum_{k=0}^{M-1} \frac{(M\rho)^k}{k!} + \frac{(M\rho)^M}{M!} \cdot \frac{1}{1-\rho}\right)^{-1} , \tag{2.19}$$

其中，M 表示服务器的数量。当所有的服务器都处于忙的状态时，用户新提交的服务请求必须等待并插入到上述 FCFS 队列中。如果用P_q表示由于没有空闲服务器，新到达的服务请求需要排队的概率，则有：

$$P_q = \sum_{k=M}^{\infty} p_k = \frac{p_M}{1-\rho} = p_0 \frac{(M\rho)^M}{M!} \cdot \frac{1}{1-\rho} \quad , \tag{2.20}$$

假设用N表示处于等待或执行中的平均服务请求数量，可得：

$$\bar{N} = \sum_{k=0}^{\infty} k p_k = M\rho + \frac{\rho}{1-\rho} P_q \quad . \tag{2.21}$$

已有的研究文献中，有很多不同的衡量云服务质量的标准[46]。在绝大部分衡量服务质量的标准中，基于用户服务请求的响应时间来衡量服务质量是一个使用最广泛的方法。与已有研究类似，本文同样基于用户服务请求的响应时间来衡量服务质量。响应时间的定义为从释放到完成所需要的时间，包括执行时间以及等待时间。根据 Little 法则[47]，用$\bar{R}$表示用户服务请求的平均响应时间，则有：

$$\bar{R} = \frac{\bar{N}}{\lambda_2} = \bar{x_1}(1 + \frac{P_q}{M(1-\rho)}) = \bar{x_1}(1 + \frac{p_M}{M(1-\rho)^2}) \quad , \tag{2.22}$$

根据公式(2.22)，可知用户服务请求的平均等待时间为:

$$\bar{W} = \bar{R} - \bar{x_1} = \bar{x_1} \frac{p_M}{M(1-\rho)^2} \quad . \tag{2.23}$$

用户服务请求的等待时间是影响用户服务满意度的最主要原因。云服务运营商应该将用户服务请求的等待时间尽可能的降低，可以通过例如增加服务器数量或者加快服务器的执行速度等方式来降低用户服务请求在队列中的等待时间。一旦等待时间超过 SLA 协议中所规定的限制阈值则应该对用户进行补偿，这就是本文所提出的低服务质量收费惩罚模型的出发点。

2.3.2 收入模型

云服务运营商的收入主要来源于对用户提交的服务请求按使用量进行收费。本文假定在正常收费周期t内一旦确定用户的需求之后云服务的价格保持不变，但是第四章提出的动态自适应闭环控制系统模型依然能够保证不同收费周期内动态定价策略的实现。同样为了便于研究，我们认为每个服务请求对应一个用户。因此收费周期内，实际购买用户数量的期望就是实际的服务请求数。

不失一般性，我们假定每一个用户的服务请求消耗同样数量的资源，包括CPU，内存，存储以及带宽等[30]。假设用Revenue表示收费周期t内云服务运营商的收入，则有：

$$\text{Revenue} = \omega \cdot E_{\omega}[m] \ , \tag{2.24}$$

其中，ω表示每个服务请求的收费；$E_{\omega}[m]$表示定价为ω时实际购买的服务请求数量的期望。

2.3.3 功耗模型与电费支出

CMOS 数字电路中的功耗和电路延迟可以通过简单的方程模拟，即使对于复杂的微处理器电路来说也是如此。CMOS 电路的功耗分为动态和静态功耗两种；然而在一个设计良好的电路中，动态功耗占据主导地位。假设用P_d表示动态功耗，则 CMOS 电路的动态功耗可以表示为$P_d = aCV^2f$；其中，a,C分别表示动态活动因子和载入电容，V表示供给电压，f表示时钟频率[48]。理想情况下，供给电压与时钟频率的关系为$V \propto f^{\varphi}$，其中常量$\varphi > 0$[49]。处理器的执行速度s通常与时钟频率呈线性关系，即$s \propto f$。为了方便讨论，本文假定$V = bf^{\varphi}$且$s = cf$，其中b和c是常量因子。因此动态功耗可以表示为$P_d = aCV^2f = ab^2Cf^{2\varphi+1} = (ab^2C/c^{2\varphi+1})\, s^{2\varphi+1} = \xi s^{\gamma}$；其中，$\xi = ab^2C/c^{2\varphi+1}$，$\gamma = 2\varphi + 1$。例如，当设定$b = 1.16$，$aC = 7.0$，$c = 1.0$，$\varphi = 0.5$，$\gamma = 2\varphi + 1 = 2.0$，$\xi = ab^2C/c^{\gamma} = 9.4192$，此时通过$P_d = aCV^2f = \xi s^{\gamma}$计算出的$P_d$的值，与 Intel Pentium M 处理器的实际功耗极其接近[50]。

本文考虑两种模式下的服务器速度和功耗模型，即 idle-speed 模式和 constant-speed 模式。在 idle-speed 模式下，当没有任务执行时服务器运行速度为0，即此时只有静态功耗。由于速度为s时的功耗为ξs^{γ}，因此单位时间内每个服务器的平均动态能耗为$\rho\xi s^{\gamma} = \frac{\lambda_2}{M}\bar{r}\xi s^{\gamma-1}$。多服务器系统中 M 个服务器单位时间内的平均动态能耗为$M\rho\xi s^{\gamma} = \lambda_2\bar{r}\xi s^{\gamma-1}$。然而，只要服务器处于开启状态，即使没有任务执行它仍然具有一定的功耗，例如静态功耗和漏电流功耗[49]。我们用

P_s表示当服务器处于空闲状态时最基本的功耗，用P表示 M 个服务器单位时间内的总功耗，因此$P = M(\rho P_d + P_s) = \lambda_2 \bar{r} \xi s^{\gamma-1} + MP_s$。

在 constant-speed 模式下，所有的服务器都运行在速度s下，尽管此时没有任务执行。同样，用P表示 M 个服务器单位时间内的总功耗。由于速度为s时的功耗为ξs^{γ}，因此$P = M(P_d + P_s) = M(\xi s^{\gamma} + P_s)$。

关于云服务运营商的电费支出，本文采用大部分研究文献中使用的实时电价模型。对于实时电价模型，不同时间段的费用不同，例如高峰低谷期就有明显不同的收费。用$C(x_2)$表示实时电价模型中的收费函数，其中x_2表示收费周期t内多服务器系统的总能耗。用P_{total}表示表示收费周期t内多服务器系统的总功耗，其与P成线性相关。用Bill表示收费周期t内，云服务运营商应缴纳的电费，则有：

$$\text{Bill} = C(P_{total}). \tag{2.25}$$

2.3.4 租赁费用模型

云服务运营商从基础设施提供商处租赁服务器资源构建自己的云计算环境。例如，每台服务器都需要通信宽带，域名，路由器，服务器本身等软硬件资源。不失一般性且为了方便讨论，我们用δ表示收费周期t内每台服务器所需的平均租赁费用，该费用包括构建一台完整服务器所需全部资源的费用。假设用Rent_{cos}表示收费周期t内所需的租赁费用，则有：

$$\text{Rent}_{cos} = \delta M, \tag{2.26}$$

其中，M 是服务器的个数。

2.3.5 对弹性用户的奖励模型

本文假定每一个用户的服务请求消耗同样数量的资源，包括 CPU，内存，存储以及带宽。用$\lambda_2^i[\tau']$表示i类型用户在第τ'个时隙内的平均到达率，用$\tau = t/T$表示收费周期t内的时隙数量。由于用户提交的服务请求设定的延迟时隙D_i不同，所获得的奖励也不同。假设用Reward表示整个收费周期t内云服务运营商对用户的奖励，则有：

$$\text{Reward} = \sum_{\tau'=1}^{\tau}\sum_{i=1}^{I}(\hbar_i - L_i)\lambda_2^i[\tau']T, \tag{2.27}$$

其中，$\hbar_i$表示i类型用户由于其工作负载的灵活性所获得的预期奖励，L_i表示用户的收入损失，分别参见公式(2.13)与(2.12)；T表示每个时隙的长度。值得注意的是，对于第τ'个时隙，单位时间内的总服务请求到达率为：$\lambda_2 = \sum_{i=1}^{I}\lambda_2^i[\tau']$。

实际上，关于用户的收入损失因子κ_i，云服务运营商可以通过历史数据或者经验分布获得。我们假定κ_i在云服务请求被调度前已经知晓。通过用公式(2.16)代替公式(2.13)中的D_i，可将用户的预期奖励$\hbar_i$表示如下：

$$\hbar_i = \theta log(1 + max(min(\frac{\theta}{\kappa_i} - 1, D_{max}), 0)). \tag{2.28}$$

2.3.6 低服务质量的收费惩罚模型

在多服务器模型(2.3.1)一节中已经指出用户服务请求的响应时间是影响用户服务满意度的最主要原因。一旦响应时间超过 SLA 协议中所规定的限制阈值则应该对用户进行补偿，从云服务运营商的角度来看则是对其低服务质量时所进行的收费惩罚。这就是本文提出低服务质量收费惩罚模型的动机。

用户服务请求的响应时间与服务质量密切相关，关系到对用户的服务满意度。由于低服务质量时需要对用户进行收费补偿，所以也会影响运营商的利润。为了研究关于低服务质量时的收费惩罚模型，我们需要一个完整的费用补偿机制。其中涉及到服务请求的执行需求，服务等级协议 SLA，用户的满意度，服务的质量。

如果所有的服务器运行速度固定为s，则对于执行需求(待执行指令数)为r的用户服务请求来说，其执行时间为$x = r/s$。假定用s_0表示服务器运行的基准速度，作为 SLA 中达到用户服务满意度的参考值。对于i类型的用户，基于其服务请求的待执行指令数r与响应时间$\overline{R}$，本文定义如下的低服务质量收费惩罚函数：

$$u_i(r,\bar{R})=\begin{cases}0, & 0\le \bar{R}\le \dfrac{r}{s_0}+D_i \\ d(\bar{R}-\dfrac{r}{s_0}-D_i), & \dfrac{r}{s_0}+D_i<\bar{R}\le(1+\dfrac{\omega}{d})\dfrac{r}{s_0}+D_i \\ \omega, & \bar{R}>(1+\dfrac{\omega}{d})\dfrac{r}{s_0}+D_i\end{cases}, \tag{2.29}$$

上述函数定义的理论意义如下：

1) 对于i类型的用户，如果处理其服务请求的响应时间$\bar{R}\le r/s_0+D_i$，那么云服务运营商就认为本次以高服务质量成功处理服务请求，因此不应该对用户进行收费补偿，即补偿为0。其中参数s_0表示用户的期望和满意度，这是因为$\bar{R}=r/s_0+D_i$代表正常完成用户服务请求且无超时情况发生的最长截止时间。

2) 如果处理i类型用户服务请求的响应时间大于r/s_0+D_i但是不大于$(1+\omega/d)r/s_0+D_i$，那么云服务运营商认为服务请求以低服务质量处理完成，并且对于用户的补偿应随着响应时间$\bar{R}$的增加而线性增加。参数d表示违反服务等级协议SLA时所受的惩罚程度。之所以选择$\left(1+\frac{\omega}{d}\right)\frac{r}{s_0}+D_i$作为上限，是因为$\bar{R}=\left(1+\frac{\omega}{d}\right)\frac{r}{s_0}+D_i=\frac{r}{s_0}+D_i+\frac{\omega}{d}\cdot\frac{r}{s_0}$，所以我们加上惩罚因子$d$，并给了重新执行用户服务请求的机会。如果执行完成则认为是低服务质量，否则服务请求的执行超时。

3) 如果处理i类型用户服务请求的响应时间$\bar{R}$大于$(1+\omega/d)r/s_0+D_i$，那么云服务运营商认为此次的服务请求等待了太长时间，因此不应该对用户进行收费，也即是补偿费用为用户购买服务的价格ω。

用Penalty表示收费周期t内，对于所有的服务请求应该给予的补偿，则有：

$$\text{Penalty}=\sum_{\tau'=1}^{\tau}\sum_{i=1}^{I}u_i(r,\bar{R})\lambda_2^i[\tau']T, \tag{2.30}$$

其中，τ表示收费周期t内的时隙个数，T表示每个时隙的长度，$\lambda_2^i[\tau']$表示i类型用户的服务请求在第τ'个时隙内的平均到达率。

值得注意的是，实际服务请求的响应时间$\overline{R}$是与具有速度s_0(即基准或参考速度)的服务器上的任务执行时间进行比较，从而应用上述的惩罚函数。每个服务器的实际运行速度是由云服务运营商决定，可以高于也可以低于基准速度s_0。这取决于系统工作负载(即λ_2与$\overline{r}$)、系统参数(例如，M, α, γ, P等)、对用户的奖励(例如，θ, κ_i)以及低服务质量时的惩罚函数(例如d等)。基于这些，本文将求取最优的多服务器配置，使得后文定义的利润函数可以得到最大值。

基于上述介绍的多服务器模型以及相应的收入耗费模型，我们将云服务运营商的利润定义为收入减去各种支出费用，包括租赁成本、电费、对弹性用户的奖励、低服务质量时的惩罚。用Profit表示收费周期t内云服务运营商利润，则有：

$$\text{Profit} = \text{Revenue} - \text{Bill} - \text{Rent}_{\cos} - \text{Reward} - \text{Penalty},$$

其中，上述利润表达式右边的每一部分分别表示：收入、电费、租赁费、对用户的奖励、低服务质量时的惩罚费用。本文下一步将会基于上述利润表达式，给出详细的利润最大化问题的定义并提出相应求解利润最大化问题的算法。

2.4 本章小结

本章主要工作是对本文后续研究所用到的架构及模型进行详细介绍。首先介绍了典型的三层云计算架构模型，其中主要包含三类角色：提供基础服务设施的供应商，购买基础设施服务并搭建云计算环境的云服务运营商以及购买云服务的用户；其次，介绍了用户模型，主要包括用户服务感知价值，用户的需求分布以及对弹性用户的MR奖励返现模型；然后，介绍了云服务运营商模型，对其中的多服务器模型，收入与耗费模型，对弹性用户的奖励以及低服务质量时的惩罚模型进行了详细的介绍；最后，对求解的目标问题进行了简单介绍。本文后续讨论的问题定义以及求解最优解算法的设计都是基于本章的模型。

第三章 最优服务定价与系统配置的研究

上一章节详细介绍了本文涉及到的系统模型，主要包括用户需求分布模型及云服务运营商的收入与耗费模型。基于这些模型，本部分研究基于用户感知价值的最优服务定价、最优系统运行参数以及相关求解算法的设计与分析。本章首先定义所求解的最优化问题及其相关约束条件；由于带约束条件的原问题不是凸函数，然后本文设计算法将其转化为若干个凸子域并构建子最优化问题；最后设计算法求解各个子域的最优解，基于子域的解得到原问题的全局最优解。

3.1 问题定义

如章节 2.2 中用户的需求分布模型所述，云服务的定价直接影响购买服务的用户数量。换句话说，云服务可以看作是一种同样遵循价值规律的特殊商品。随着云服务价格的显著上升，用户需求的数量急剧下降，反之亦然。虽然较低的价格可以吸引大量的用户，但由于各种耗费支出，最终利润可能低于云服务运营商的预期。另一方面，较高的价格导致较少的用户，最终利润可能仍低于预期。

本文旨在最大化云服务运营商的利润，同时保证用户最大的服务满意度，具体包括两种措施：根据用户设置的服务请求可允许延迟的时隙长度对其进行奖励，低服务质量时对用户进行收费补偿。我们将云服务运营商的利润定义为收入减去各种支出费用，包括租赁成本、电费、对弹性用户的奖励、低服务质量时的惩罚。利润最大化问题表示如下：

$$\text{Profit} = \text{Revenue} - \text{Bill} - \text{Rent}_{\cos} - \text{Reward} - \text{Penalty}, \tag{3.1}$$

其中，上述最优化问题等式右边的每一部分分别表示：收入、电费、租赁费、对用户的奖励、低服务质量时的惩罚，且分别由公式(2.24)(2.25)(2.26)(2.27)(2.30)定义。接下来，我们列举云服务运营商做一系列决策时所面临的操作约束条件。

首先，奖励因子必须是非负的：

$$\theta \geq 0 .$$

第二，收费周期t内的多服务器集群的能耗应该有个上限。根据前述所讲，服务器的能耗与其运行速度有直接的关系。为了求得最优的系统运行速度，我们需要优化云服务运营商的电费支出部分，即降低多服务器集群的能耗。这个在实际过程中是存在的，比如拉闸限电措施等。用P_{th}表示P的上限。由于能耗与功耗程线性相关，所以我们用功耗来表示这种约束，表示如下：

$$P \le P_{th} \ .$$

第三，由于用户提交的服务请求可以根据需要自行设定最长延迟调度时隙长度，所以在每个时隙内被延迟的服务请求的量必须非负并且不超过新到达服务请求的数量。对于i类型的用户，在第τ'个时隙内，假设用$\phi_i[\tau']$表示其服务请求被延迟的数量。因此有如下约束：

$$0 \le \phi_i[\tau'] \le \lambda_2^i[\tau']T \ ,$$

其中，$\lambda_2^i[\tau']$表示第τ'个时隙内i类型用户服务请求的到达速率，T表示每个时隙长度。显然，每个时隙内被调度的服务数量为$(\lambda_2^i[\tau'] - \phi_i[\tau'])$。由于用户提交服务的服务请求可以允许延迟调度，所以对于i类型的用户来说，从收费周期开始直到任何时隙产生的服务请求量必须不小于在该时段内被调度的请求量，因此有:

$$\sum_{\tau'=1}^{\tau}(\lambda_2^i[\tau']T - \phi_i[\tau']) \le \sum_{\tau'=1}^{\tau}\lambda_2^i[\tau']T, \forall i,\tau \ ,$$

其中，τ表示收费周期t内时隙的数量；由于T表示每个时隙的长度，则有$\tau = t/T$。

第四，基于服务等级协(SLA)，设定响应时间上限。用户无法接受过长响应时间而得到的结果。用R_{th}表示响应时间上限，则有：

$$\bar{R} \le R_{th} \ .$$

最后，i类型用户服务请求的调度时间不应该晚于其设定的允许延迟时隙长度D_i。对于每个时隙内的服务请求，从时隙开始到得到调度，i类型用户服务请求的最晚调度时间不应该超过$T + \lfloor D_i \rfloor$，其中$\lfloor \cdot \rfloor$表示向下取整。因此，有如下约束条件：

$$\sum_{\tau'=1}^{\tau+\lfloor D_i \rfloor}(\lambda_2^i[\tau']T - \phi_i[\tau']) \ge \sum_{\tau'=1}^{\tau}\lambda_2^i[\tau']T, \forall i,\tau \ ,$$

值得注意的是，每个时隙内i类型用户服务请求的最大延迟时隙$T + \lfloor D_i \rfloor < R_{th}$。

本文中，我们认为云服务运营商在每个计费周期开始时优化其决策，具体包括云服务定价、用户的需求分布、系统运行所需参数。因此，本文寻求解决以下问题：

$$\text{maximize:} \quad \text{Profit} \tag{3.2}$$

$$\text{subject to:} \quad \theta \geq 0 \tag{3.3}$$

$$P \leq P_{th} \tag{3.4}$$

$$0 \leq \phi_i[\tau'] \leq \lambda_2^i[\tau']T \tag{3.5}$$

$$\bar{R} \leq R_{th} \tag{3.6}$$

$$\sum_{\tau'=1}^{\tau+\lfloor D_i \rfloor} (\lambda_2^i[\tau']T - \phi_i[\tau']) \geq \sum_{\tau'=1}^{\tau} \lambda_2^i[\tau']T, \forall i, \tau \tag{3.7}$$

上述最优化问题的最优解不但与多服务器的数量和运行速度有关，而且与用户的服务满意度及服务等级协议 SLA 相关。因此我们的问题变成了选择最优的服务定价，最优的工作服务器数量及其运行速度，最优的奖励因子等系统配置使得云服务运营商的利润最大。同时由于奖励函数(由公式(2.28)表示)及约束条件(由公式(3.7)表示)的限制，导致最优化问题(由公式(3.2)表示)并不是一个凸函数问题，因此求解比较困难。下一小节我们着重求解并不呈现凸函数性质的利润最大化问题。

3.2 分解算法

本节我们设计了一个算法用来求解由奖励函数定义和相关约束条件引起的非凸最优化问题，主要分为三步。首先，我们提出了一个分解算法将原最优化问题进行分解。具体来说，我们迭代的更新关于奖励因子θ的子域，每更新一次θ的子域就构建一个与该子域对应的子优化问题，从而将原问题迭代的分解为与θ的每个子域相关的子最优化问题。可以证明按照本文的算法进行分解，每个子域都可以视为一个凸集。然后，在每一个θ的子域中，我们提出了一个基于增广拉格朗日方法的算法，用此算法求解每个子域中的子最优化问题，并保存每个子域中的最优解。最后，遍历所有子域，选择能使得利润最大化的那个子域，其所包含的各参数的解即为我们所需的全局最优解。

本文通过迭代划分θ子域及相应的数学推导和证明，表明当θ在每个子域中取其下界时，不但可以获得该子域对应子最优化问题的最优解，还能使得对应子最优化问题呈现凸函数性质。本文将这种通过划分子域在子优化问题的最优解中寻找原始问题全局最优解的算法称为分解算法。分解算法的求解过程基于如下两个定理。

【定理 1】对于每一个θ的子域θ_{sub}，该子域对应的延迟时隙D_i的最优取值为：

$$\lfloor D_i \rfloor = D_i^{L_b},\ \forall \theta_{sub} \in [L_b,\ Ub - \epsilon], \forall i \in I\ ,$$

其中，$\lfloor D_i \rfloor$表示D_i向下取整；$D_i^{L_b}$表示用户服务请求允许延迟时隙长度D_i的最优取值(由公式(3.8)表示)；L_b表示每个θ子域中θ的下界，U_b表示每个θ子域中θ的上界；ϵ表示一个非常小的趋近于 0 的正常数。定理 1 的证明参见附录[B-1]。

【定理 2】对于每一个θ的子域θ_{sub}，该子域对应的子最优化问题(由公式(3.13)表示)中优化变量θ的最佳取值是$\theta = L_b$。定理 2 的证明参见附录[B-2]。

下面详细的介绍上述分解算法的工作流程，我们将由公式(3.2)表示的最优化问题迭代地划分如下：

1. 用L_b表示每个θ子域中θ的下界，初始化θ子域的下界。基于约束条件$\theta \geq 0$，设置θ的下界L_b的取值为$L_b = 0$。
2. 基于公式$D_i = \max(\min(\frac{\theta}{\kappa_i} - 1,\ D_{max}), 0)$，计算每个$\theta$的子域中$D_i$的下界$D_i^{L_b}$，与$L_b$有关。$D_i^{L_b}$可以表示为：

$$D_i^{L_b} = \left\lfloor max\left(min\left(\frac{L_b}{\kappa_i} - 1,\ D_{max}\right),\ 0\right)\right\rfloor,\ \forall i \in I. \tag{3.8}$$

3. 用U_b表示每个θ子域中θ的上界，按如下确定其上界U_b：

$$U_b = \begin{cases} \infty\ , & \Theta = \emptyset \\ min_{i \in \Theta}\{(D_i^{L_b} + 2)\kappa_i\}\ , & \text{其它} \end{cases}\ , \tag{3.9}$$

其中，域Θ的定义如下：

$$\Theta = \{i \in 1, \cdots, I | L_b < (D_{max} + 1)\kappa_i\}\ . \tag{3.10}$$

4. 得到θ的一个新子域，如下：

$$L_b \leq \theta_{sub} \leq U_b - \epsilon\ , \tag{3.11}$$

其中，ϵ表示一个非常小的趋近于 0 的正数。根据定理 1，可取$\lfloor D_i \rfloor = D_i^{L_b}$。然后用上述步骤2 中的$D_i^{L_b}$(由公式(3.8)表示)代替由公式(3.7)所表示的约束条件中的$\lfloor D_i \rfloor$，并构建一个新的 affine 约束条件：

$$\sum_{\tau'=1}^{\tau+D_i^{L_b}} (\lambda_2^i[\tau']T - \phi_i[\tau']) \geq \sum_{\tau'=1}^{\tau} \lambda_2^i[\tau']T, \forall i, \tau . \tag{3.12}$$

接下来，根据上述子域的新约束条件，构建相应θ子域对应的子最优化问题：

$$\text{maximize:} \quad \text{Profit} \tag{3.13}$$

$$\text{subject to:} \quad L_b \leq \theta_{sub} \leq U_b - \epsilon$$

$$P \leq P_{th}$$

$$0 \leq \phi_i[\tau'] \leq \lambda_2^i[\tau']T$$

$$\overline{R} \leq R_{th}$$

$$\sum_{\tau'=1}^{\tau+D_i^{L_b}} \left(\lambda_2^i[\tau'] - \phi_i[\tau']\right) \geq \sum_{\tau'=1}^{\tau} \lambda_2^i[\tau']T, \quad \forall i, \tau$$

5. 上述子最优化问题(由公式(3.13)表示)也并不是一个凸问题。根据定理 2，子最优化问题中的变量θ在每个子域的最佳取值是其下限，即$\theta = L_b$。很容易证明用$\theta = L_b$带入$\hbar_i = \theta \log(1 + D_i)$(由公式(2.13)表示)中进行计算，可以使得每个子域对应的子最优化问题呈现为一个具有凸性质的优化问题。
6. 基于步骤 5，设置$\theta = L_b$。用本文提出的基于增广拉格朗日方法的算法求解与该θ子域相关的最优化问题，即子最优化问题。然后保存该子域的解。
7. 如果$U_b < \infty$则更新L_b，设置$L_b = U_b$并回到步骤 2 遍历与θ相关的另一个子域；如果$U_b = \infty$，表明已经迭代遍历了θ的所有子域，即所有可能的子优化问题。因此，我们可以在这些子优化问题的最优解中找到原始问题的全局最优解。
8. 遍历所有的子域，从所有子优化问题的最优解中找到利润最大的那一组解作为全局最优解，该子域的所有参数作为系统运行的最优配置参数。

上述分解算法的伪代码形式如算法 1 所示。在算法 1 中，输入为每次迭代时用来求解每个θ子域中子最优化问题的算法，由函数 Solution(loop, θ_{sub})表示；以及用来保存每个子域最优解的变量 Result[loop]，其中 loop 表示第 loop 次迭代。输出

则为全局最优解。算法首先进行相关变量的初始化(Line 1~2)。Line 3~19 是算法主体部分，主要是进行循环迭代划分θ子域并求解每个子域的最优解。其中，Line 4~7 对应上述步骤 1-5，主要是划分子域以及构建每个子域的子最优化问题；Line 8~9 则是用来求解并保存每个子域的解；Line 10~16 则是用来在迭代划分子域结束时获得最优解，如果满足迭代结束条件(Line 10)，则迭代结束并选出全局最优解(Line 11~14)；Line 17 用来更新每个θ子域中θ的下界，而 Line 18 则是用来进行迭代计数，方便保存每次迭代子域的解。

算法 1：子域分解算法

Input: 在第 loop 次循环时求解每个子域最优解的算法 Solution(loop, θ_{sub})；
保存每个循环的最优解的变量 Result[loop]
Output:全局最优解,即$max\{\text{Result}[i], \forall i \in [1,\text{loop}]\}$

/*初始化循环控制变量 loop*/
1. loop = 1
/* 用L_b表示θ子域的下界，并根据公式(31)初始化L_b为 0 */
2. $L_b = 0$
/* 迭代划分θ子域开始，并求解每个子域对应的子最优化问题 */
3. **while** 1 **do**
/* 第一步：构建子域及子域中的优化问题和相应的约束条件 */
4. 计算第 loop 次循环对应的θ子域的上界，如公式(3.8)(3.9)(3.10)所示
5. 获得第 loop 次循环对应的θ的一个子域$\boldsymbol{\theta_{sub}}$，如公式(3.11)所示
6. 构建新的 affine 约束条件，如公式(3.12)所示
7. 构建与该子域对应的子最优化问题，如公式(3.13)表示

/* 第二步：求解子域中子优化问题的最优解 */
/* 具体求解算法如算法 2 中的 Solution(loop，θ_{sub})方法所示 */
/* 求解第 loop 次循环对应的子最优化问题(由公式(3.13)表示) */
8. **Solution(loop, $\boldsymbol{\theta_{sub}}$)**
/* 保存第 loop 次循环对应的子域中的解在一个全局变量中 */
9. Result[loop] = **Solution(loop, $\boldsymbol{\theta_{sub}}$)**
/* 如果$U_b = \infty$，则表示已经遍历了θ的所有子域，迭代结束 */
10. **if** $U_b = \infty$ **then**
11. **for** i **in** [1, loop]
/* 返回利润最大的那个子域的下标 */
12. j = max{Result[i].Profit}
/* 返回利润最大的那个子域对应的解 */

```
13.                return Result[j]
14.            end for
15.            break
16.        end if
           /* 迭代更新θ的下界 */
17.        L_b = U_b
           /* 迭代次数加 1，继续下一个子域的迭代 */
18.        loop = loop + 1
19.    end while                    //迭代结束
```

图 3-1 子域分解算法

3.3 求解最优服务定价与系统配置参数

上一节介绍了子域分解算法，将目标问题迭代的分解成与奖励因子θ相关的子域，在每个子域中我们可以得到θ的最优取值，相应子域中的求解目标转换成求解具有凸性质的子最优化问题。虽然每个子域对应的是具有凸性质的子最优化问题，但是由于子最优化问题及其相应约束条件的复杂性，常规的凸优化知识并不能求得最优解。因此，本节我们基于增广拉格朗日方法，提出了一个求解最优服务定价与最优系统配置参数的算法，我们称为扩展的增广拉格朗日—EALA(Extended-Augmented-Lagrangian algorithm)算法。

算法 1 给出了子域划分方法并且构建了对应子域的子最优化问题，如公式(3.13)所示。由定理 2 可知，θ在每个子域中的最优取值是$\theta = L_b$。与θ相关的只有利润表达式中的 Reward 部分与公式(3.8)-(3.11)表示的约束条件，由于θ此时取固定值，与θ相关的约束条件均变成了常量，因此子最优化问题可以转化如下：

$$\text{maximize:}\omega E_{\omega}[m]-C(P_{total})-\delta M-\sum_{\tau'=1}^{\tau}\sum_{i=1}^{I}(\hbar_i-L_i)\lambda_2^i[\tau']T-\sum_{\tau'=1}^{\tau}\sum_{i=1}^{I}u_i(r,\bar{R})\lambda_2^i[\tau']T$$

$$\text{subject to: } \begin{aligned} &P=\lambda_2\bar{r}\xi s^{\gamma-1}+MP_s\le P_{th} \quad \text{(idle-speed 模式)}\\ &P=M(\xi s^{\gamma-1}+P_s)\le P_{th} \quad \text{(constant-speed 模式)}\\ &\bar{R}=\bar{x}_1(1+\frac{p_M}{M(1-\rho)^2})\le \bar{R}_{th} \end{aligned}, \tag{3.14}$$

上式中各变量及表达式的含义均已在第 2 章中进行过详细的介绍；其中，P_{th}，R_{th}分别表示功耗和响应时间的阈值，为常数。

为方便计算，将问题(3.14)中的R_{th}、P_{th}分别转化为用b_1、b_2表示的数学符号；$E_{\omega}[m]$用ω相关的函数$h(\omega)$表示，其中$h(\omega)= \alpha\beta t(1-F(\omega))$；由于$C(P_{total})$与功耗有关，并且总能表示一个与常数的乘积，因此采用$C(P_{total})=b_3P$表示。值得注意的是，根据泰勒展开式有：$\sum_{k=0}^{M-1}\frac{(M\rho)^k}{k!}\approx e^{M\rho}$，$M!\approx\sqrt{2\pi M}\left(\frac{M}{e}\right)^M$；所以公式(2.18)与(2.20)可分别化简为：

$$p_M=\frac{1-\rho}{\sqrt{2\pi M}(1-\rho)\left(\frac{e^{\rho}}{e\rho}\right)^M+1}$$

$$P_q=\sum_{k=M}^{\infty}P_k=\frac{P_M}{1-\rho}=\frac{1}{\sqrt{2\pi M}(1-\rho)(e^{\rho}/e\rho)^M+1}$$

因此，由公式(3.14)表示的子优化问题可转化成如下带约束的形式：

$$\begin{cases}O(\omega,M,s)=\omega\cdot h(\omega)-b_3P-\delta M-\sum_{\tau'=1}^{\tau}\sum_{i=1}^{I}(\hbar_i-L_i)\lambda_2^i[\tau']T-\sum_{\tau'=1}^{\tau}\sum_{i=1}^{I}u_i(r,\bar{R})\lambda_2^i[\tau']T\\ g_1(M,s)=b_1-\bar{x}_1(1+\frac{P_M}{M(1-\rho)^2})\geq 0\\ g_2(M,s)=b_2-P\geq 0\end{cases},(3.15)$$

其中，功耗$P=\begin{cases}\lambda_2\bar{r}\xi s^{\gamma-1}+MP_s\ ,\ (\text{idle}-\text{speed 模式})\\ M(\xi s^{\gamma}+P_s)\quad ,\ (\text{constant}-\text{speed 模式})\end{cases}$，$g_1(M,s)$与$g_2(M,s)$分别表示与变量$M$、$s$相关的函数。

基于上述介绍，现在我们的目标变为求解由公式(3.15)表示的带不等式约束的最优化问题，即maximize：{公式(3.15)}。为方便后续算法求解，我们先求解公式(3.15)中各表达式关于ω、M、s的偏导数以及构建增广拉格朗日函数，然后按照如下步骤进行迭代计算。

步骤 1：关于最优云服务定价，求解关于服务定价ω的偏导。

对$h(\omega)$关于服务价格ω求偏导：

$$\frac{\partial h(\omega)}{\partial\omega}=-\alpha\beta tf(\omega).$$

对$O(\omega,M,s)$关于服务定价ω求偏导：

$$\frac{\partial O(\omega,M,s)}{\partial \omega}=\frac{\partial h(\omega)}{\partial \omega}\cdot\omega+h(\omega)=\alpha\beta t\big(1-\omega f(\omega)-F(\omega)\big) \quad (3.16)$$

其中，$f(\omega)$、$F(\omega)$分别表示在ω处的概率密度和累积分布函数。

步骤 2：优化最优服务器数量，求解关于服务器数量M的偏导。

对$g_1(M,s)$关于M求偏导：

$$\frac{\partial g_1(M,s)}{\partial M}=\frac{\partial}{\partial M}\left[-\frac{\overline{x_1}}{M}\left(\frac{1}{\left[\sqrt{2\pi M}(1-\rho)(e^{\rho}/e\rho)^{M}+1\right](1-\rho)}\right)\right]+\frac{\overline{x_1}}{M^2}\frac{p_M}{(1-\rho)^2} \quad (3.17)$$

为方便继续求解，令：

$$\begin{aligned}\mathrm{D}_1&=\sqrt{2\pi M}(1-\rho)(e^{\rho}/e\rho)^{M}+1\\&=\sqrt{2\pi M}(1-\rho)L+1\ ,\end{aligned}$$

其中，$L=(e^{\rho}/e\rho)^{M}$；

注意到，根据公式$\rho=\lambda_2/M\cdot\bar{r}/s$(由公式(2.17)表示)，有$\frac{\partial\rho}{\partial M}=-\frac{\lambda_2\bar{r}}{M^2 s}=-\frac{\rho}{M}$；

对L两边同取对数，可得：

$$\ln L=M(\rho-\ln\rho-1)$$

对上式$\ln L$两边同取关于M的偏导数，则有：

$$\frac{1}{L}\frac{\partial L}{\partial M}=(\rho-\ln\rho-1)+M\left(1-\frac{1}{\rho}\right)\frac{\partial\rho}{\partial M}$$

根据上式，继续求解D_1关于M的导数：

$$\begin{aligned}\frac{\partial D_1}{\partial M}&=\sqrt{2\pi}\left(\frac{1}{2\sqrt{M}}(1-\rho)L+\sqrt{M}\left(-\frac{\partial\rho}{\partial M}\right)L+\sqrt{M}(1-\rho)\frac{\partial L}{\partial M}\right)\\&=\sqrt{2\pi}\left(\frac{1}{2\sqrt{M}}(1+\rho)L-\sqrt{M}(1-\rho)\ln\rho L\right)\ .\end{aligned}$$

不妨令$D_2=1-\rho$；对D_2关于M求偏导，有$\frac{\partial D_2}{\partial M}=-\frac{\partial\rho}{\partial M}=\frac{\rho}{M}$；

将上述D_1,D_2关于M的偏导数带入式子(3.17)，继续对$g_1(M,s)$关于M求偏导：

$$\frac{\partial}{\partial M}g_1(M,s)=\frac{\overline{x_1}}{MD_1D_2}\left[\frac{\frac{\partial D_1}{\partial M}D_2+\frac{\partial D_2}{\partial M}D_1}{D_1D_2}+\frac{1}{M}\right]\ . \quad (3.18)$$

对$g_2(M,s)$求关于M的偏导：

$$\frac{\partial g_2(M,s)}{\partial M}=\begin{cases} P_s & ,\ \text{idle}-\text{speed 模式} \\ \xi s\gamma+P_s & ,\ \text{constant}-\text{speed 模式} \end{cases}. \tag{3.19}$$

步骤 3：优化服务器运行速度，求解关于服务器运行速度s的偏导。

根据公式$\rho=\lambda_2/M\cdot\ \bar{r}/s$，有$\frac{\partial\rho}{\partial s}=-\frac{\lambda_2\bar{r}}{Ms^2}=-\frac{\rho}{s}$；同样，由于$L=\ (e^{\rho}/e\rho)^M$，则$L$关于$s$的偏导数为：

$$\frac{1}{L}\frac{\partial L}{\partial s}=M\left(1-\frac{1}{\rho}\right)\frac{\partial\rho}{\partial s}=\frac{M}{s}(1-\rho)$$
$$\Rightarrow\frac{\partial L}{\partial s}=\ \frac{M}{s}(1-\rho)L$$

对D_1关于s求偏导：

$$\frac{\partial D_1}{\partial s}=\sqrt{2\pi M}\left[\left(-\frac{\partial\rho}{\partial s}L+(1-\rho)\frac{\partial L}{\partial s}\right)\right]$$
$$=\ \sqrt{2\pi M}[\rho+M(1-\rho)^2]\frac{L}{s}$$

对D_2关于s求偏导：

$$\frac{\partial D_2}{\partial s}=-\frac{\partial\rho}{\partial s}=\frac{\rho}{s}$$

继续求解$g_1(M,s)$ 对于s的偏导：

$$\frac{\partial}{\partial s}g_1(M,s)=-\frac{\overline{x_1}}{M}\cdot\frac{\partial}{\partial s}\left(\frac{p_M}{(1-\rho)^2}\right)$$
$$=-\frac{\overline{x_1}}{M}\left[\frac{\frac{\partial D_1}{\partial s}D_2+\frac{\partial D_2}{\partial s}D_1}{(D_1D_2)^2}\right]. \tag{3.20}$$

对$g_2(M,s)$求解关于s的偏导：

$$\frac{\partial g_2(M,s)}{\partial s}=\begin{cases} \lambda_2\bar{r}\xi(\gamma-1)s^{\gamma-2} & ,\quad \text{idle}-\text{speed 模式} \\ M\xi\gamma s^{\gamma-1} & ,\quad \text{constant}-\text{speed 模式} \end{cases} \tag{3.21}$$

步骤 4：依据增广拉格朗日方法，定义增广拉格朗日函数。

将公式(3.15)表示的带约束条件的最优化问题转化为如下等式约束问题的形式：

$$\Phi(\omega,M,s,\boldsymbol{y},\boldsymbol{v},\sigma)=O(\omega,M,s)-\sum_{j=1}^{2}v_j(g_j(M,s)-y_j^2)+\frac{\sigma}{2}\left(g_j(M,s)-y_j^2\right)^2$$

其中，向量$\boldsymbol{y},\boldsymbol{v}$分别表示修正函数与修正因子；$\sigma>0$，为常量参数；

用配方法将上式转化为：

$$\Phi(\omega,M,s,\boldsymbol{y},\boldsymbol{v},\sigma)=O(\omega,M,s)+\sum_{j=1}^{2}[\frac{\sigma}{2}\left[y_j^2-\frac{1}{\sigma}\left(\sigma g_j(M,s)-v_j\right)\right]^2-\frac{v_j^2}{2\sigma}]$$

将上述用配方法转化后的函数$\Phi(\omega,M,s,\boldsymbol{y},\boldsymbol{v},\sigma)$关于$y_j$取极大值，得：

$$y_j^2=\frac{1}{\sigma}max\left(0,\sigma g_j(M,s)-v_j\right),j=1,2$$

将上式带回原式，由此可以定义拉格朗日增广函数：

$$\Phi(\omega,M,s,\boldsymbol{y},\boldsymbol{v},\sigma)=O(\omega,M,s)+\frac{1}{2\sigma}\sum_{j=1}^{2}[\left[max(0,v_j-\sigma g_j(M,s))\right]^2-v_j^2] \quad (3.22)$$

步骤 5：对修正因子向量$\boldsymbol{v}$进行修正。

按如下修正公式进行修正因子v_j的迭代修正：

$$v_j^{l+1}=\max\left(0,v_j^{(l)}-\sigma g_j\left(M^{(l)},s^{(l)}\right)\right), \quad (3.23)$$

其中，$v_j^{(l)},M^{(l)},s^{(l)}$分别表示相应变量的第$l$次的迭代值；

定义迭代收敛函数$Q_j(M,s)=g_j(M,s)-y_j^2,j=1,2$，以

$$\| Q_j\left(M^{(l+1)},s^{(l+1)}\right)\|/\| Q_j\left(M^{(l)},s^{(l)}\right)\|$$

来衡量收敛快慢；

步骤 6：具体增广拉格朗日乘子法迭代过程如下：

a) 给定初始点$M^{(0)}$，$s^{(0)}$，乘子向量初始估计$v^{(1)}$，参数σ，允许误差$\varepsilon>0$，常数$\eta>1$，任选常量$\Psi\in(0,1)$，置初始迭代次数$l=1$；

b) 以$M^{(l-1)}$，$s^{(l-1)}$为初点，根据公式(3.16)-(3.21)相关变量的偏导数，求解无约束问题，即maximize: {公式(3.22)}：

$$\max\ \Phi(\omega,M,s,\boldsymbol{y},\boldsymbol{v},\sigma),$$

得第l次迭代的解$\omega^{(l)}$，$M^{(l)}$，$s^{(l)}$；

c) 若$\| Q_j\left(M^{(l)},s^{(l)}\right)\|<\varepsilon$，则停止计算，得到最优解$\omega^{(l)}$，$M^{(l)}$，$s^{(l)}$；否则进行步骤 d)；

d) 若

$$\frac{\| Q_j(M^{(l+1)}, s^{(l+1)}) \|}{\| Q_j(M^{(l)}, s^{(l)}) \|} \geq \Psi \ ,$$

则置$\sigma = \eta\sigma$；

e) 否则，无法进行修正，用公式(3.23)中的修正公式进行修正因子的修正，计算v_j^{l+1}，并置$l = l + 1$，然后回到步骤 b)继续计算。

上述求解算法的伪码表示形式如算法 2 所示。算法分为两个主要函数：第一个是 Solution(loop, θ_{sub})，该函数根据算法 1 中划分的关于奖励因子θ的每个子域，构建增广拉格朗日函数，然后迭代求解所构建的无约束问题；第二个函数是 unconstr ($\Phi(\omega, M^{(l)}, s^{(l)}, y, v, \sigma)$, constr_count)，该函数由 Solution(loop, θ_{sub})函数调用，用来具体求解构建好的无约束问题，并返回所需解。Line 1~8 是相关变量的初始化工作；此外，算法还定义了最大迭代次数(Line 8~9)。Line 9~29 是迭代求最优解的过程。首先，算法判断是否是第一次迭代(Line 10)，如果是则以初始迭代点进行迭代计算，然后调用自定义函数unconstr(*)求解由公式(3.22)表示的无约束问题(Line 10~11)；否则对于第l次迭代使用第$(l-1)$次迭代所得的解进行计算，调用unconstr(*)求解由公式(3.22)表示的无约束问题(Line 12~13)。接下来，如果所求得的解满足迭代收敛条件，则保存此次的解为最优解并退出循环(Line 15~19)；否则更新拉格朗日乘子或者进行修正因子的修正(Line 20~23)。最后，返回最优解包括最优定价ω，最优工作服务器数量M，最优运行速度s以及最优解对应的利润(Line 29)。Line 30~37 定义了函数unconstr(*)，用来求解由公式(3.22)表示的无约束问题。Line 33~35 主要是令公式(3.16)-(3.21)中关于ω, M, s的偏导数等于 0，求解无约束问题。

算法 2：EALA 算法：求解最优服务定价与系统配置

Input: θ 的子域θ_{sub}；
迭代初始点$M^{(0)}, s^{(0)}$，乘子向量初始估计$v^{(1)}$, 参数$\sigma > 1$；
允许误差ε，常数$\eta > 1$, 任选常量$\Psi \in (0,1)$, 初始迭代次数l

Output: 每个子域的最优解，包括最优定价ω，最优工作服务器数量M，最优运行速度s以及最优解对应的利润；结果保存在Result[θ_{sub}]变量中

/*函数定义 */

Solution(loop, θ_{sub}):

/* 初始化赋值操作 */

1. $v^{(1)} = 1$ //初始拉格朗日乘子
2. $\varepsilon = 0.005$ //误差控制范围
3. $\eta = 2$ //常数
4. $\Psi = 0.3$ //任选常量
5. $l = 1$ //初始迭代次数置 1
6. $\sigma = 2$ //参数σ，惩罚系数
7. max_itera = 25 //最大迭代次数
8. constr_count = 2 //约束条件个数

/* 算法迭代开始 */

9. **while** $l <$ max_itera **do**
10. **if** $l == 1$ **then** //如果是第一次迭代

/* 求解由公式(3.22)表示的无约束问题，调用函数**unconstr** */

11. $[\omega^{(1)}, M^{(1)}, s^{(1)}]$ = **unconstr**($(\Phi(\omega, M^{(0)}, s^{(0)}, \boldsymbol{y}, \boldsymbol{v}, \sigma)$, constr_count)
12. **else**

/* 求解无约束问题，得第l次迭代解，调用函数**unconstr** */

13. $[\omega^{(l)}, M^{(l)}, s^{(l)}]$ = **unconstr**($(\Phi(\omega, M^{(l-1)}, s^{(l-1)}, \boldsymbol{y}, \boldsymbol{v}, \sigma)$, constr_count)

/* 判断停止条件 */

14. **for** $j = 1:$ constr_count **do**
15. **if** $\| Q_j(M^{(l)}, s^{(l)}) \| < \varepsilon$ **then** // 若满足上述步骤 c)
16. res $= [\omega^{(l)}, M^{(l)}, s^{(l)}]$ //第l次迭代解即为所求解
17. Profit $\leftarrow$ res //求解最优解对应的利润

/* 记录最优解 */

18. Result$[\theta_{sub}] =$ [Profit, $\omega^{(l)}, M^{(l)}, s^{(l)}$]
19. **break**

/* 若满足上述步骤 d) */

20. **else if** $\| Q_j(M^{(l+1)}, s^{(l+1)}) \| / \| Q_j(M^{(l)}, s^{(l)}) \| \geq \Psi$ **then**
21. $\sigma = \eta\sigma$ //更新拉格朗日乘子
22. **else**

/* 用公式(48)中的修正公式进行修正 */

23. $v_j^{l+1} = \max(0, v_j^{(l)} - \sigma g_j(M^{(l)}, s^{(l)}))$
24. **end if**
25. **end for**
26. **end if**
27. $l = l + 1$ // 更新迭代次数

28. **end while** // 迭代结束
29. **return** Result[θ_{sub}] // 返回本次最优解

/* 定义求解无约束问题的函数，该函数用来求解转化后的无约束问题 */
30. **procedure unconstr** ($\Phi(\omega, M^{(l)}, s^{(l)}, y, v, \sigma)$, constr_count)
// 基于无约束的拟牛顿法求解
31. **for** $j = 1$: constr_count **do**
32. 根据公式(3.16)-(3.21)中关于ω, M, s的偏导数，求解无约束问题
33. $\omega \leftarrow \frac{\partial}{\partial \omega}\Phi(\omega, M^{(l)}, s^{(l)}, y_j, v_j, \sigma) = 0$ //基于公式(3.16)
34. $M \leftarrow \frac{\partial}{\partial M}\Phi(\omega, M^{(l)}, s^{(l)}, y_j, v_j, \sigma) = 0$ //基于公式(3.17)(3.18)(3.19)
35. $s \leftarrow \frac{\partial}{\partial s}\Phi(\omega, M^{(l)}, s^{(l)}, y_j, v_j, \sigma) = 0$ //基于公式(3.20)(3.21)
36. **end for**
37. **return** $[\omega, M, s]$ //返回无约束问题的解

图 3-2 求解最优服务定价与系统配置的 EALA 算法

3.4 本章小结

本章主要是求解最优云服务定价与最优的多服务器系统运行时配置参数。本章首先定义了利润最大化问题的数学表示形式，并给出了相应的约束条件，这些约束条件包括：奖励因子的非负要求，基于 SLA 的响应时间要求，用户服务请求在其设定的延迟时隙上限D_i前得到调度的约束，多服务器集群的功耗约束。然后，本章提出了一个分解算法，用来求解由于某些表达式和约束条件导致的非凸最优化问题；具体来说是将原问题迭代的分解成一个个与奖励因子θ子域对应的子问题，并求解每个子问题的最优解，通过子问题的最优解获得原问题的全局最优解。算法 1 详细描述了分解算法的工作过程。最后，本章详细介绍了本文所提出的扩展的增广拉格朗日——EALA 算法，用来具体求解每个子问题的最优解。

本章所考虑的利润最大化问题涉及到用户服务请求的数量，服务请求的紧急程度(是否允许延迟执行)，应用环境的工作负载，多处理器集群的配置，服务等级协议(SLA)，服务质量(QoS)，用户的服务满意度，低服务质量时的惩罚，对弹性服务请求的返现奖励，多服务器集群租赁费用，能耗费用，市场环境中用户的动态需求

分布等因素。通过设计分解算法和 EALA 算法求解相关变量的最优取值，从而使得云服务运营商能够基于这些最优参数配置获得最大利润。

第四章 动态自适应闭环控制系统的研究

上一章主要讨论了云服务运营商利润最大化问题的定义，以及求解最优云服务定价与最优多服务器系统运行配置参数的相关算法设计。由于随着市场环境的改变，用户的需求分布也会相应改变。因此，为了获得最大利润与较高的用户服务满意度，云服务的定价也不应该保持恒定不变。基于用户的动态需求变化，本章介绍一个动态自适应闭环控制系统(Dynamic Adaption Loop Control System, DALCS)模型。应用此模型，既可以满足用户的动态需求变化获得较高用户服务满意度，又可以使得云服务运营商能动态调整每个收费周期的最优服务定价和多服务器系统运行参数，同时系统也能循环稳定的运行。

4.1 动态自适应闭环控制系统模型概述

本节我们介绍动态自适应闭环控制系统(DALCS)模型。所谓系统的动态自适应是指：一个系统能根据运行状态或条件的改变而动态地调整自身的运行设置，使得系统本身始终运行在最优的状态下；而所谓的闭环控制是指：整个系统运行在一个无线循环的模式中，然后根据动态自适应性调整运行配置。实际的需求环境并不是一成不变，一旦外界市场环境中某个条件改变（例如竞争者推出的商品定价，电价，租赁费等等），用户的需求及云服务运营商的相应耗费都会改变。系统在正常运营时不能随意关停，因此需要设计一种能控制系统根据需要动态做出改变，重新获取最优的服务定价及多服务器系统运行配置的方案。

外界商品的价格经常随市场环境改变，相应的对于云服务这种特殊的商品来说，其价格也会根据不同时段的市场条件的改变而改变。由于用户的需求分布与云服务定价密切相关，因此价格的改变会带来相应的用户感知价值的改变。用户的感知价值反过来又会影响用户对某个定价下云服务的需求量。另一方面，除了用户的需求分布会通过影响服务购买量来影响云服务运营商的利润之外，电价、租赁费的改变也会导致利润的改变。因此，动态自适应闭环控制系统模型需要从用户的动态需求分布和运营商的动态耗费支出两方面进行设计。鉴于此，本文提

出了一个基于用户动态感知价值分布和运营商动态耗费两种可变因素的动态自适应系统模型。下一节也是综合考虑上述两种可变因素来进行动态自适应闭环控制算法的设计，算法的设计思想如下。

每一次价格变动，在进行相应的云服务定价时，我们都希望制定的云服务价格符合真实情况下的用户需求分布规律。这样，制定的服务价格才能不违背市场环境下用户的需求定律。通过统计云服务定价以及与之相关的用户需求的历史数据，得出用户感知价值的概率分布。根据预测出的用户感知价值分布规律，进行用户需求分布的建模(参见 2.2 小节)，预测该定价下的云服务购买量。此外，如果电费、租赁费等耗费部分参数改变，还需重新计算云服务运营商的耗费。然后基于预测的下一阶段用户的需求分布和新的云服务运营商的耗费，重新求解利润最大化问题(参见第三章)，从而获得新的定价下的最优云服务定价及最优系统运行配置。基于动态自适应闭环控制系统模型，整个系统的工作流程如图4-1所示。

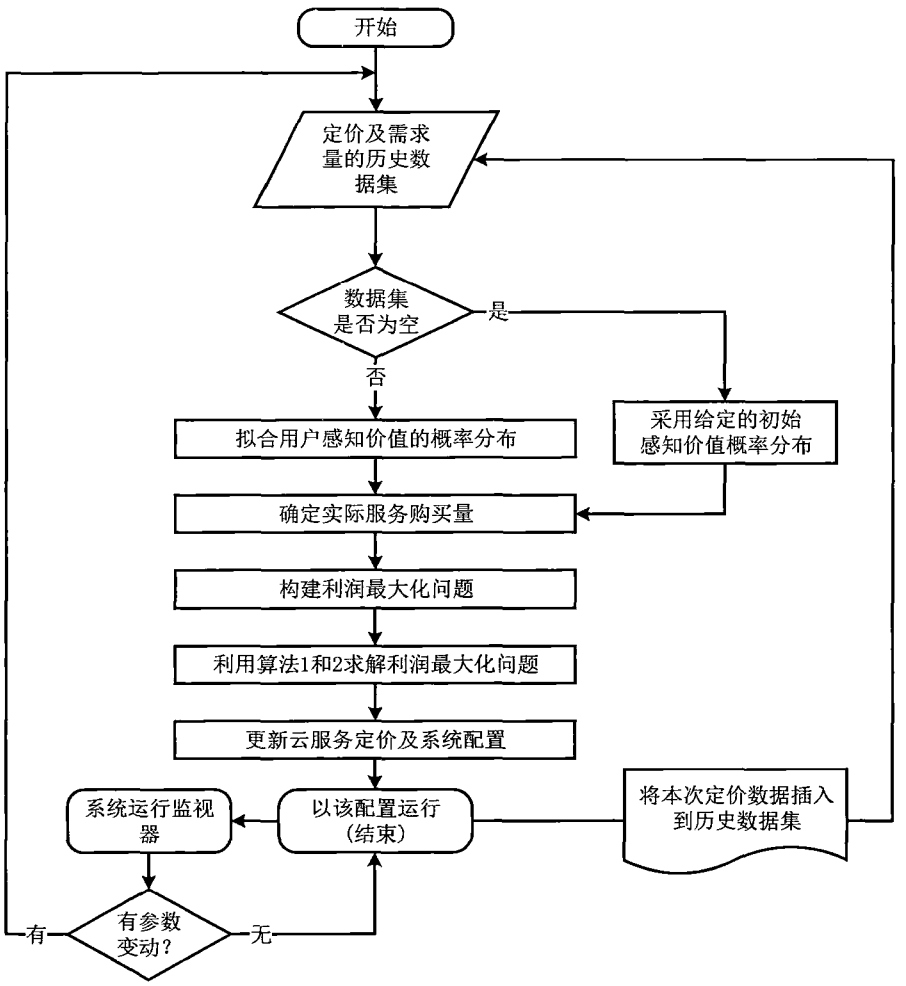


图 4-1 动态自适应闭环控制系统(DALCS)工作流程图

下面简单介绍图 4-1 所表示的动态自适应闭环控制系统工作流程图。首先，本文建立了一个历史数据集合，用来存储历史的服务定价和与该定价相关的用户需求。通过这些历史定价与需求数据，我们可以拟合出用户感知价值的概率分布，为下次定价和预测用户感知价值分布以及与需求分布提供参考。

当系统第一次部署运行时，则历史数据集合为空，那么我们会根据经验分布给出一个初始的感知价值概率分布。然后基于该初始感知价值概率分布计算利润最大化问题。然后把最优解，包括最优定价及最优系统运行配置，储存到历史数据集合中。当在某一次求解利润最大化的过程中(非第一次部署系统)，如果历史数据集合不空，那么基于该集合中的数据，拟合出用户的感知价值概率分布。然后基于该感知价值概率分布重新构建并计算利润最大化问题，得出最优解，同时将最优解储存到历史数据集合中，系统以该最优解中的相关参数运行。

本文还为整个系统设置了一个系统运行监视器，用来监测系统是否有参数改变(例如物价、电费，租赁费等的改变)。一旦检测到系统有参数变动，则当前应用的服务定价以及系统运行配置并不一定能使得云服务运营商利润最大化。所以，此时会重新计算最优服务定价与系统配置参数。

整个系统工作在一个循环模式下，一旦外界有参数变动则会触发动态自适应，重新更新并计算利润最优化问题。基于所求的最优解更新服务定价与系统运行配置，然后继续由系统运行监视器监视是否有参数变动，决定是否重新求解最优化问题。整个系统工作在这种动态自适应闭环控制模式中。

4.2 动态自适应闭环控制系统的算法设计

上一小节介绍了云服务定价系统中采用动态自适应闭环控制模型的必要性及其设计思想。通过动态自适应模型，可以保证整个系统能动态地根据历史数据学习以及适应外界条件的改变；通过闭环控制，整个系统可以持续平稳的运行下去。下面将详细介绍动态自适应模型的 DAA(Dynamic Adaption Algorithm)算法。DAA 算法流程图如图 4-2 所示。

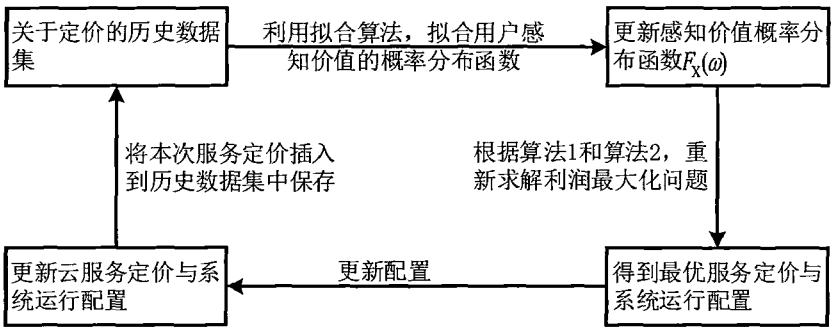


图 4-2 DAA 算法流程图

下面我们详细介绍 DAA 算法流程：

第一步，每一次求解利润最大化的问题时，如果历史定价数据集不空，那么 DAA 算法将会根据历史定价数据，拟合出用户的感知价值概率分布。根据第 2 章用户需求模型可知，感知价值分布直接影响着用户的需求数量，因此直接影响着云服务运营的收入；

第二步，得到新的感知价值概率分布之后，我们可以重新预测未来一段时间用户感知价值的概率分布并获得用户的需求分布。如果此时系统还有其他参数的改变，例如电费等，那么同样需要更新利润最大化表达式中的相应部分：

$$\text{Profit}_{\text{new}} = \text{Revenue} - \text{Bill} - \text{Rent}_{\cos} - \text{Reward} - \text{Penalty},$$

然后重新求解利润最大化问题，具体求解算法那参见算法 1 与算法 2；

第三步，获得最优解之后则基于该解所表示的参数，云服务运营商可以获得最大利润，用户可以获得最高的服务满意度。因此，应用本次获得的最优解进行更新，具体包括最优云服务定价，最优多服务器系统运行速度与工作服务器数量；

第四步，将定价插入到历史数据集合中保存，一旦未来系统参数发生动态改变，可以作为参考来重新预测下一阶段用户感知价值分布及相应的需求分布。

DAA 算法的伪代码表示形式如算法 3 所示。该算法的核心在于需要根据函数**ksdensity**(Ω)计算定价数据集Ω的概率密度函数。然后重新构建新的利润最大化问题 **max** $\{\text{Profit}_{\text{new}}\}$，最后根据算法 1 和算法 2 求解最优解，并将最优定价再次插入到历史数据集中以供未来参考。算法的输入包括历史定价数据集，系统运行监视器的检测结果(是否有参数变动)，子域分解算法 1 和求解最优化问题的

算法 2。算法的输出为每次动态调整后的最优定价ω，最优服务器数量M和速度s。算法工作在一个循环中(Line 1)。首先需要知道系统是否有参数变动(Line 2)，如果无参数变动则继续以当前配置参数运行(Line 3~4)，否则重新构建利润最大化问题并求解(Line 5~13)。Line 6~7 主要是根据历史定价数据构建用户感知价值的概率分布。Line 8 则是根据相应参数的改变更新利润最大化问题的表达式，即$\text{Profit}_{\text{new}}$。然后根据算法 1 和算法 2 求解最优解(Line 9~10)。最后使用本次所获得的最优解来更新云服务定价和系统运行配置，并将本次定价结果插入到历史数据集中保存(Line 11~12)。ksdensity(*)函数主要用来根据历史定价数据拟合用户感知价值的概率分布。Line 18~22 是拟合的主体过程。函数返回的是最终拟合出来的概率密度函数(Line 24)。

算法 3：DAA 算法：动态自适应算法

Input: 历史定价数据集；
系统运行监视器的检测结果—check；
算法 1，算法 2
Output:最优解—ω, M, s

```
1.  while true do
        /* 获得系统是否有参数改变 */
2.      check = 系统运行监视器的检测结果          // true 表示有参数改变
3.      if  check == false  then
4.          continue                              // 继续以当前参数运行
5.      else
6.          Ω = 历史定价
            /* 根据ksdensity(Ω)函数拟合集合Ω中定价ω的概率密度函数 */
7.          F_X(ω) ← ksdensity(Ω)
            /* 根据相应改变更新利润最大化表达式 */
8.          Profit_new = Revenue - Bill - Rent_cos - Reward - Penalty

            /* 求解利润最大化问题 */
9.          max {Profit_new}
10.         [ω, M, s] ← 用算法 1 与算法 2 求解
11.         用本次最优解[ω, M, s]更新云服务定价与系统配置参数
12.         将ω插入历史定价数据集中
13.     end if
```

```
14. end while

    /* 函数 ksdensity(Ω)，利用核平滑法计算概率密度估计 */
15. procedure ksdensity(Ω)
16.     n = length(Ω)                          // Ω中样本的个数
17.     h= std(Ω)*(4/3/n)^(1/5)                // 西尔弗曼经验法则
18.     for x in Ω
19.         phi = @(x) (exp (-.5*x.^2)/sqrt (2*pi))   // 正态分布概率密度函数
20.         ksden = @(x) mean (phi((x-Ω)/h)/h)        // 核密度
21.         pdf[x] = fplot (ksden, [-25,25], 'k')     // 计算概率密度
22.         F_X(ω) ← pdf[x]
23.     end for
24.     return F_X(ω)                          // 返回概率密度函数
```

图 4-3 动态自适应(DAA)算法

4.3 本章小结

由于外界商品的价格经常随市场环境改变，相应的云服务价格也会根据不同时段的市场条件的改变而改变，每次的定价应该都遵循用户历史感知价值的分布和用户需求定律，而不是随意定价。为了能够动态适应外界条件的改变，本章提出了一个动态自适应系统模型。

本章首先介绍了动态自适应系统设计的必要性；然后给出了基于该动态自适应系统的整个系统工作流程；最后给出了动态自适应系统的设计算法。本章从用户的动态需求分布和运营商的动态耗费支出两方面作为切入点，设计了一个动态自适应系统模型。基于该动态自适应闭环控制系统模型，多服务器的云服务定价系统可以动态地根据外界参数的改变调整最优配置，整个系统能平稳的运行下去。

第五章 实验设置与结果分析

本章设计实验来验证本文构建的基于用户感知价值的利润最大化模型。实验主要分为如下部分：首先，由于本文构建了一个基于用户感知价值的云服务定价模型，因此我们需要验证该定价模型是否符合市场条件下的用户需求定律；其次，我们验证不同工作负载下，云服务定价对云服务运营商收入的影响；然后我们验证 idle-speed 和 constant-speed 两种模式下，云服务运营商的利润与服务器工作数量和工作速度的关系；最后，将本文的最大利润与已有的方案进行对比。

5.1 实验环境设置

本文算法运行环境为 Windows 系统下的 Matlab_x64 版本，进行仿真实验所用的电脑拥有 8GB DDR4 内存，2.56GHz Intel i7 四核处理器。

本文参考文献[30]进行云服务用户类型的设置，主要分为三类：I_1, I_2, I_3。其中第一类用户I_1属于延迟敏感型，即非弹性用户；第二类用户I_2与第三类用户I_3是弹性用户，即这两种类型用户的服务请求分别允许某种程度的延迟。延迟敏感型用户I_1的需求是基于 2014 年 1 月 1 日至 1 月 31 日期间的 YouTube America 流量；弹性类型用户I_2与I_3的需求则分别基于 2014 年 1 月 1 日至 1 月 31 日的 GMaps 和 Gmail America 流量构建。图 5-1 显示了在仿真中使用的这些工作负载轨迹的快照。这里，弹性类型工作负荷构成总工作负荷的大约 50%。

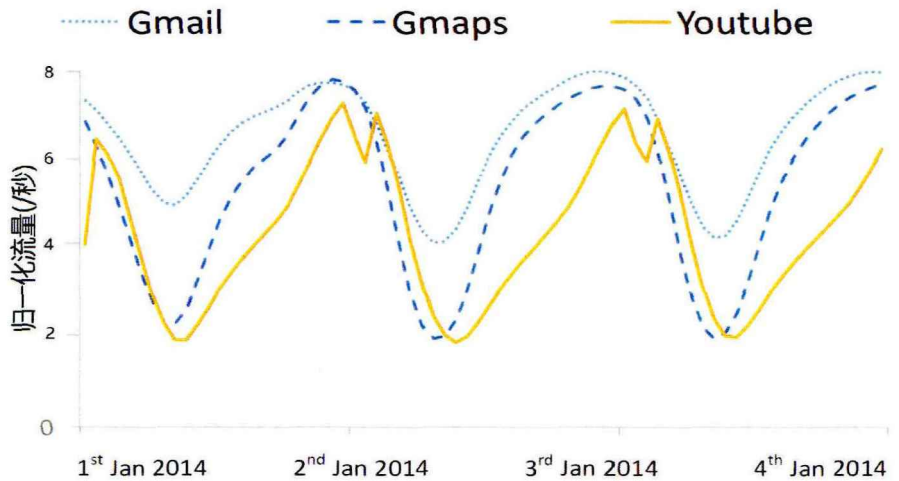


图 5-1 三种类型工作负载

电价采用实时电价计费形式[30]，其价格基于2014年1月1日至2014年1月31日的Ameren实时价格。实时电价如图5-2所示。

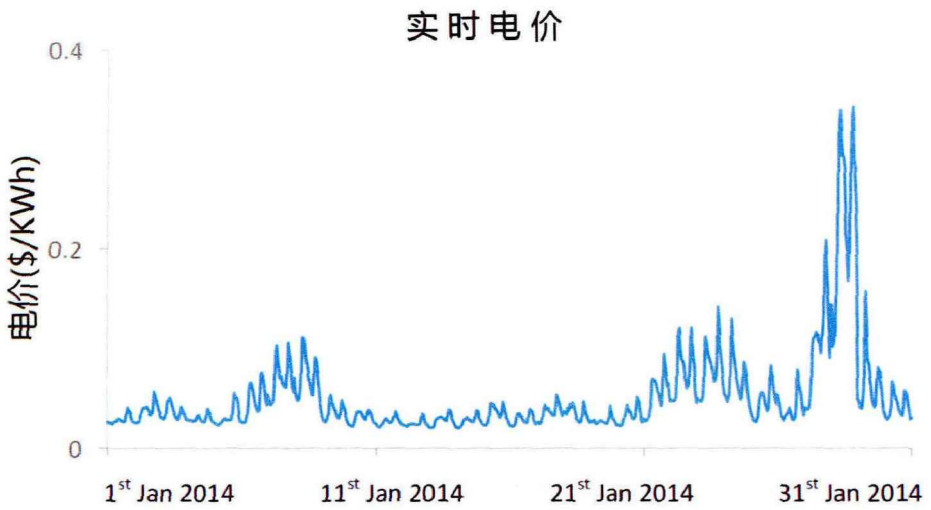


图5-2 Ameren实时电价

关于初始用户感知价值分布，本文参考文献[9]和[32]中的相关理论，假定用户感知价值服从如下正态分布：$X \sim N(0,0.22)$ 。实验中有些参数是固定的，例如收费周期、时隙长度等。将云服务收费周期t设置为30天，收费周期内的每个时隙长度T设置为$T = 1$小时。因此每个收费周期内的时隙个数为$\tau = 30 \times 24 = 720$。设置延迟时隙上限$D_{max} = 24$，$I_1, I_2, I_3$三种类型用户的收入损失因子分别为$\kappa_1 = \infty$，$\kappa_2 = 0.1$，$\kappa_3 = 0.11$。上述固定参数的设置如表5-1所示。

表5-1 实验固定参数取值表

参数符号	参数取值
t	30
T	1
τ	720
D_{max}	24
κ_1	∞
κ_2	0.1
κ_3	0.11

5.2 实验结果与分析

本节主要分析实验数据，通过实验数据来验证本文提出的定价模型符合市场环境中的用户需求定律，获取不同服务定价对用户需求和云服务运营商收入的影响，获取 idle-speed 与 constant-speed 两种不同服务器工作模式下不同工作负载对云服务运营商利润的影响，获取利润与服务器工作数量与运行速度的关系。通过与已有的利润最大化研究对比，验证本文所提利润最大化模型在不同多服务器运行配置下获取最大利润方面的优越性。

5.2.1 服务执行需求与运营商收入的结果分析

本部分主要分析用户所提交服务请求的执行需求r(即指令数)与运营商收入之间的关系。实验除了采用如表 5-1 所示的固定实验参数表外，还用到如下的实验参数，相关参数取值参考文献[2]进行设置，如表 5-2 所示。

表 5-2 验证执行需求与收入之间关系的参数表

参数名称	参数取值
平均执行需求$\bar{r}$(单位：十亿)	1
服务器数量M(单位：台)	7
基准运行速度s_0(单位：十亿/秒)	1
运行速度s(单位：十亿/秒)	1
静态功耗P_s(单位：Watt)	2
动态功耗参数 γ	2.0
动态功耗参数 ξ	9.4192
Gamma 分布参数 α	2.0
Gamma 分布参数 β	1.5

基于表 5-1 与表 5-2 中的实验参数取值，图 5-3 展示了当单位时间内的服务请求到达率$\lambda_2 = 16.15, 16.35, 16.55, 16.75, 16.95$时，云服务商运营商的收入(Revenue)与用户服务执行需求$r(0 \le r \le 3)$之间的变动关系。

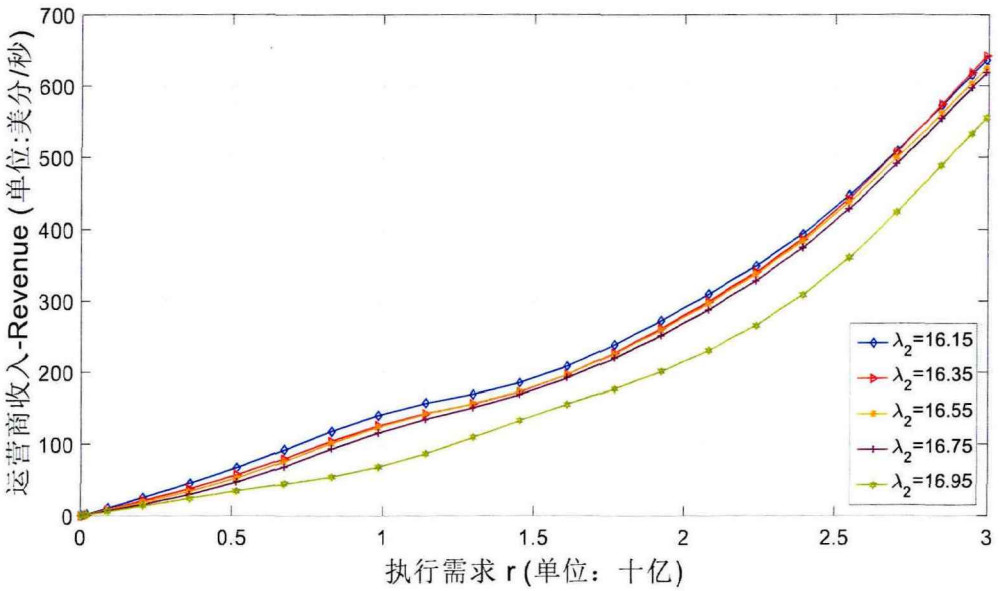
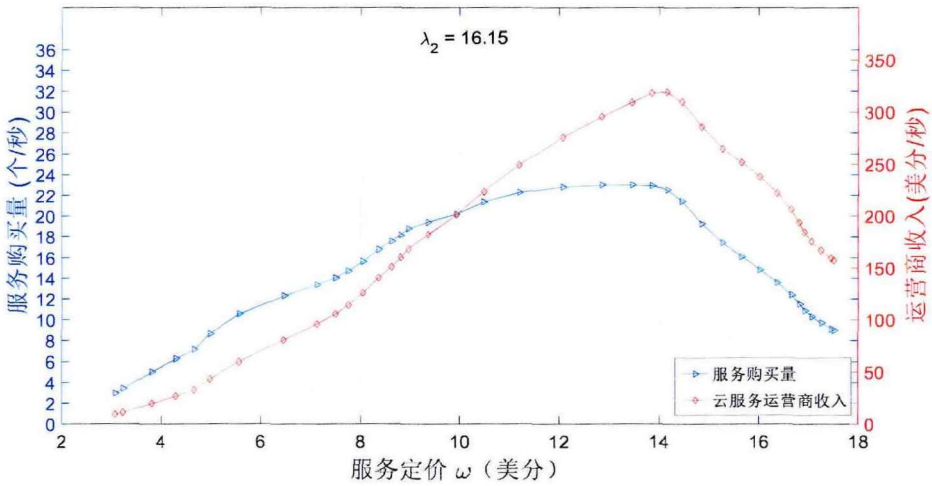


图 5-3 运营商收入与服务执行需求之间的关系

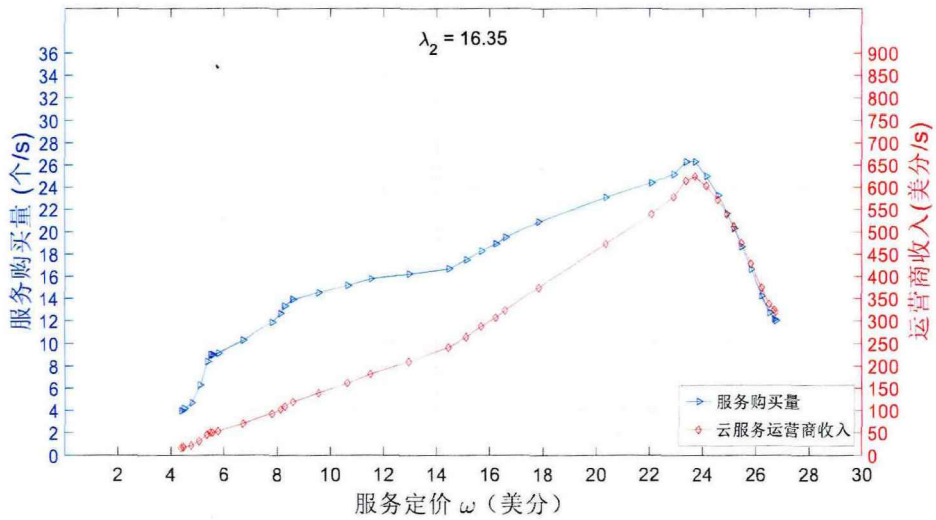
从图 5-3 中可以看出，云服务运营商收入随着服务请求到达率λ_2的增大而降低，运营商的收入是λ_2的减函数。这是因为随着λ_2的增大，多服务器系统来不及立即处理用户的所有服务请求，M/M/M 队列中等待的服务请求增多，导致用户服务请求的响应时间变高以及由此带来的低服务质量。由于低服务质量会导致对云服务运营商的惩罚费用(即 Penalty 部分)变高，因此收入降低。同样可以看出云服务运营商的收入随着执行需求r的增大而增加，这是因为与较少的用户服务请求相比，越多的服务请求意味着越多的云服务使用量，从而带来越多的服务收入。

5.2.2 定价与服务购买量及运营商收入的结果分析

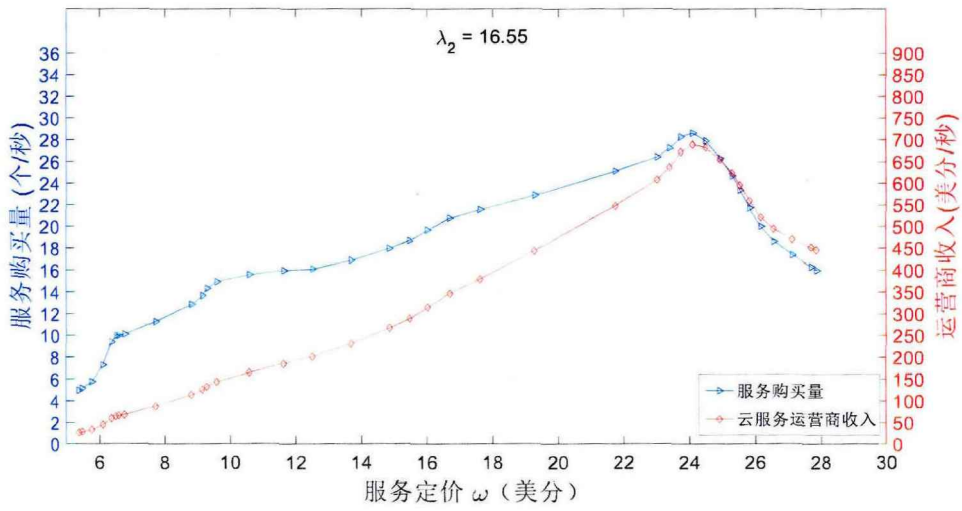
采用表 5-1 与 5-2 中的实验参数，图 5-4 展示了当用户的服务请求到达率$\lambda_2 = 16.15, 16.35, 16.55, 16.75, 16.95$时，云服务购买量以及云服务运营商的收入随云服务定价变动而变动的关系。



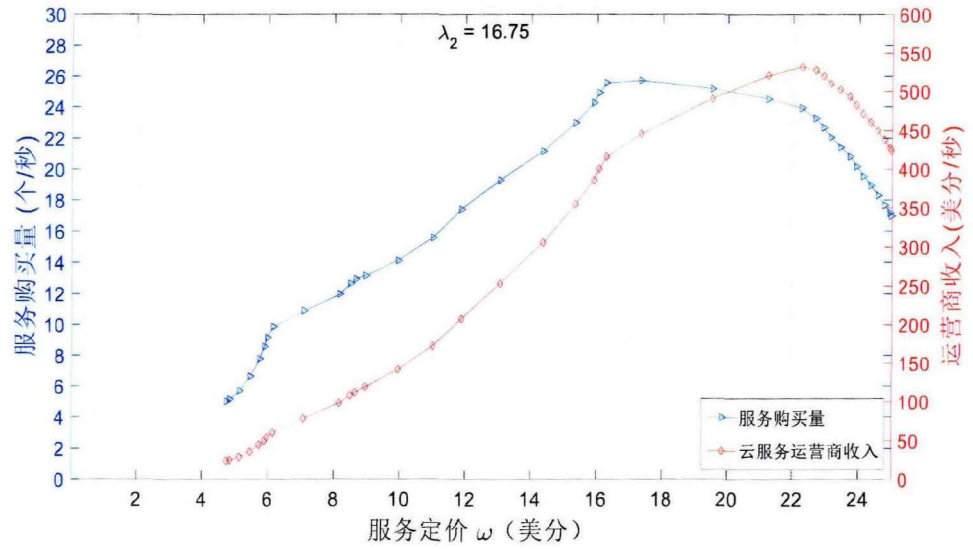
(a) $\lambda_2 = 16.15$时的服务购买量及收入



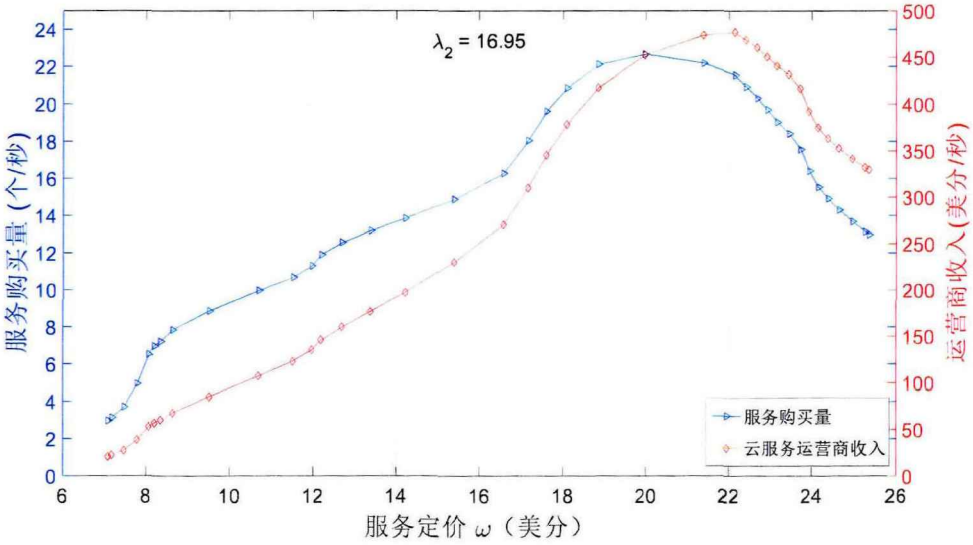
(b) $\lambda_2 = 16.35$时的服务购买量及收入



(c) $\lambda_2 = 16.55$时的服务购买量及收入



(d) $\lambda_2 = 16.75$时的服务购买量及收入



(e) $\lambda_2 = 16.95$时的服务购买量及收入

图 5-4 不同λ_2下的定价与服务购买量及运营商收入

从图 5-4 中可以很清楚的看到，在定价到达某个阈值点之前，云服务购买量逐渐增高；一旦超过阈值，则购买量会急剧下降。这比较符合实际市场环境，在实际市场环境中，当商品的价格低于用户的感知价值时，用户愿意接受此价格；然而当超出用户的感知价值，则用户的购买意愿会急剧下降。因此本文提出的基于用户感知价值的定价模型符合用户的需求定律。

关于云服务运营商的收入，从图 5-4 中可以看出，虽然在定价超过某个阈值点之后云服务的购买量会下降，然而收入却可能在随后的某个时刻达到最大值。

这也符合实际的市场环境中供需关系的数学意义，即低价格高购买量带来的收入并不一定比高价格低购买量时的收入多。当单位时间内服务到达率$\lambda_2 = 16.55$时，基于本次实验中的相关参数，云服务提供商的收入可以得到最大值。

基于以上分析，本文提出的基于用户感知价值的定价模型符合用户的需求定律，比较符合实际市场的供需环境。由于能较好的反映服务定价与用户需求以及云服务运营商收入三者之间的关系，因此本文的定价策略优于其它同类型的研究，比较具有实际意义。

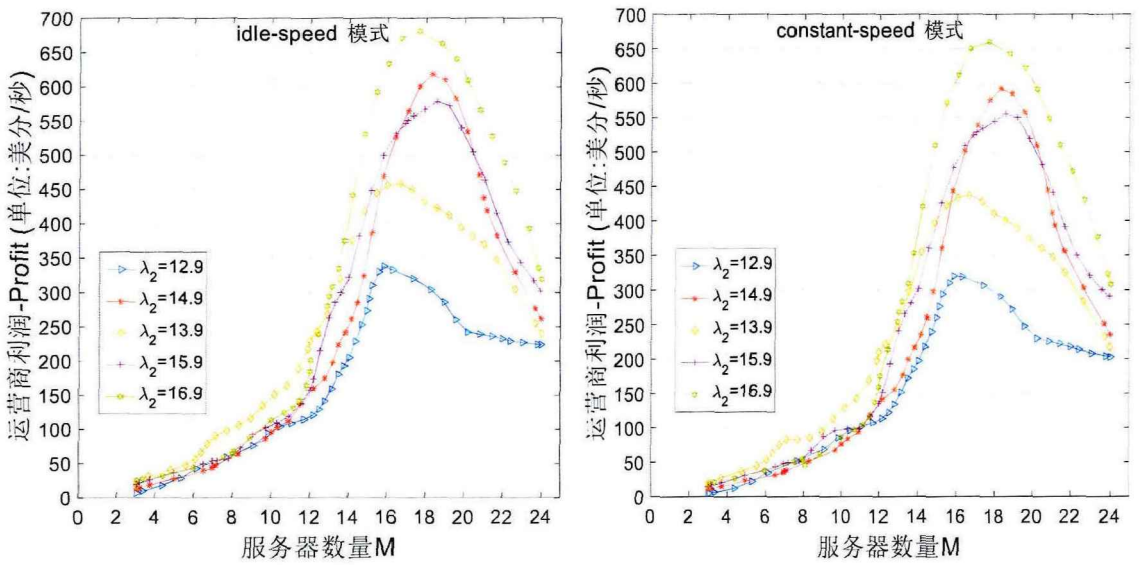
5.2.3 最优服务器工作数量与最优运行速度

本部分的实验参数除了采用如表 5-1 与表 5-2 中的参数及取值外，还需要如续表 5-3 所示的实验参数及取值。

表 5-3 实验参数续表

参数名称	参数取值
响应时间约束 b_1(单位：秒)	0.33
功耗约束 b_2(单位：Watt)	1000000
租赁费用 δ(单位：美分/秒)	1.5

采用如表 5-1 至 5-3 中所示的实验参数，图 5-5 中(a)与(b)分别展示了 idle-speed 及 constant-speed 两种模式下，当单位时间内的服务请求到达率$\lambda_2 = 12.9, 13.9, 14.9, 15.9, 16.9$时，云服务运营商的利润(Profit)与工作服务器数量M之间的变动关系示意图。



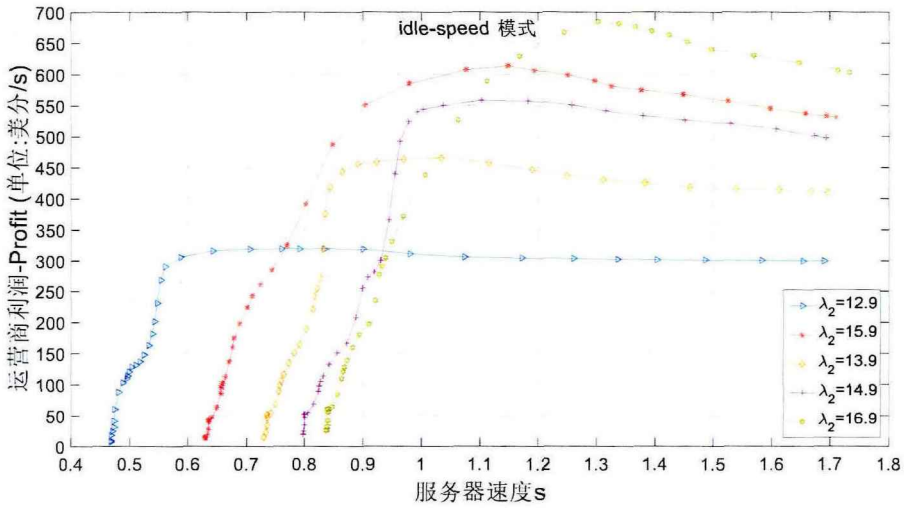
(a) idle-speed 模式 (b) constant-speed 模式

图 5-5 两种模式下利润与服务器数量之间的关系

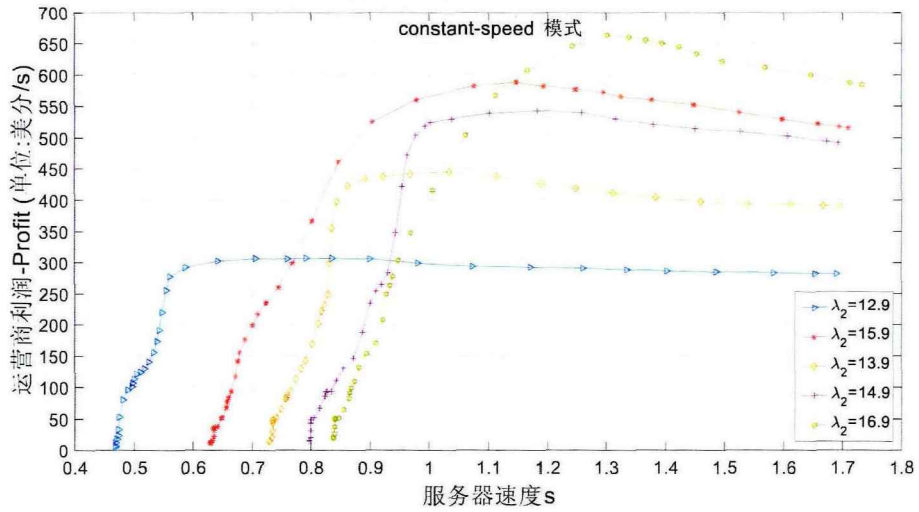
当服务请求到达率$\lambda_2 = 12.9, 13.9, 14.9, 15.9, 16.9$时，为使得云服务运营商的利润达到最大，idle-speed 服务器工作模式下的最优工作服务器数量分别是$M = 15.9124, 17.2189, 19.5605, 18.3698, 17.4857$，而 constant-speed 模式下的最优工作服务器数量分别是$M = 15.8442, 17.1168, 19.4377, 18.2848, 17.2478$。由于服务器数量需要取整，因此两者的最佳值分别都是$M = 16, 17, 19, 18, 17$。

这种最优的工作服务器数量配置具有很清晰的物理意义。当M比较小时，导致服务器工作负载$\rho \to 1$，从而导致到达的服务请求将等待较长时间。过长的响应时间会带来低服务质量，此时会增加对云服务运营商低服务质量的惩罚，因此云服务提供商的耗费增高，利润降低。随着工作服务器数量M的增加，队列中等待的服务请求数量很快降低，用户的服务请求不用等待过长的时间，惩罚耗费降低，从而利润增高。然而随着服务器数量M的持续增加，利润并没有如期增加，这是因为工作服务器数量的增加导致维护服务器运行的耗费的增加，例如电费以及租赁费用，从而利润会到达某个上限之后不会再随着服务器数量的增加而增加。

采用如表 5-1 至 5-3 中所示的实验参数，图 5-6 展示了 idle-speed 及 constant-speed 模式下，当单位时间内的服务请求到达率$\lambda_2 = 12.9, 13.9, 14.9, 15.9, 16.9$时，云服务运营商的利润(Profit)与服务器运行速度s之间的变动关系示意图。



(a) idle-speed 模式



(b) constant-speed 模式

图 5-6 两种模式下利润与服务器运行速度的关系

当服务请求到达率$\lambda_2 = 12.9, 13.9, 14.9, 15.9, 16.9$，为使利润最大，idle-speed 模式下的最优服务器运行速度分别是$s = 0.7642, 0.9435, 1.1044, 1.1293, 1.2838$；而对于 constant-speed 模式，其使得利润最大的最优服务器运行速度分别是$s = 0.7254, 0.8934, 1.0715, 1.1136, 1.2548$。

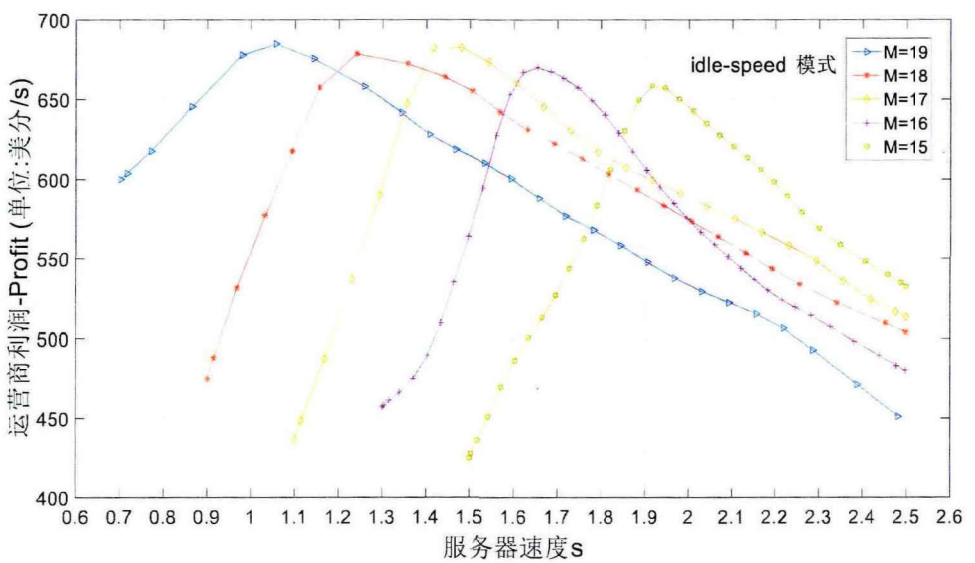
这种最优的服务器运行速度其物理意义同样比较明确。当运行速度s较小，将会导致服务器工作负载$\rho \to 1$，从而导致到达的服务请求来不及处理而将等待较长时间。因此云服务运营商不得不为其低服务质量付出代价，由于惩罚费用增高，从而导致其利润降低。随着工作服务器数量s的增加，队列中等待的服务请求很快被处理，用户的服务请求以高质量完成，对云服务运营商的惩罚降低，从而利润增高。然而随着s的持续增加，利润并没有如期增加，这是因为速度的增

加导致维护服务器运行的耗费的增加，例如能耗费用，从而利润会到达某个上限之后不会随着服务器运行速度s的增加而如期增加。

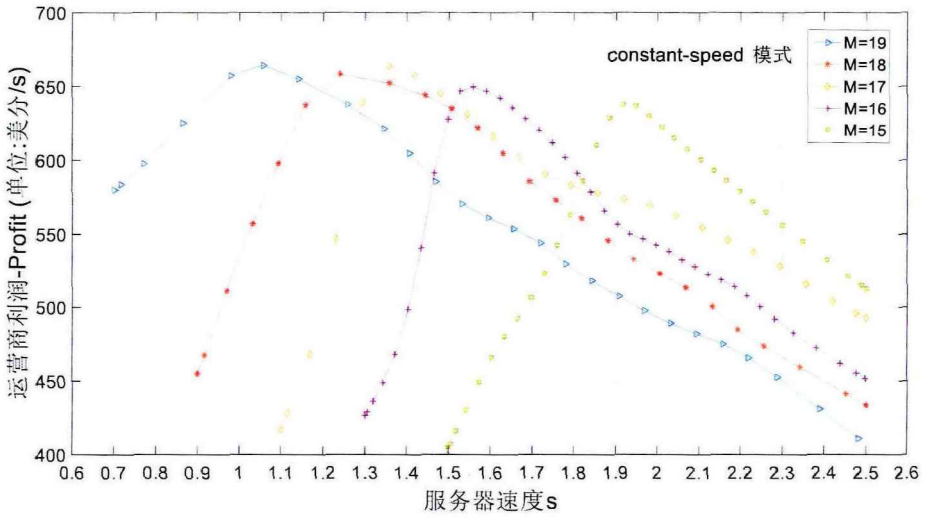
图 5-7 展示了当单位时间内的服务请求到达率$\lambda_2 = 16.9$，工作服务器数量$M = 15,16,17,18,19$且服务器运行速度s足够大时，多服务器系统在 idle-speed 及 constant-speed 两种工作模式下，使得云服务运营商利润最大化的最优工作服务器数量以及最优运行速度。

对于 idle-speed 模式，当最优配置$s = 1.4351$，$M = 16.5$时可以得到单位时间内最大的利润Profit = 687.9023美分。根据实际情况，M可取 16 或 17。当$M = 16$时，最优的运行速度为$s = 1.6237$，此时单位时间内的利润Profit = 688.8963美分；当$M = 17$时，最优的运行速度为$s = 1.4361$，此时所得利润为Profit = 689.5936美分。

对于 constant-speed 模式，当最优配置$s = 1.3826$，$M = 16.7$时可以得到单位时间内的最大的利润Profit = 668.8236美分。根据实际情况，M可取 16 或 17。当$M = 16$时，最优的运行速度为$s = 1.5316$，此时利润Profit = 665.9863美分；当$M = 17$时，最优的运行速度为$s = 1.3274$，此时单位时间内的利润Profit = 667.7962美分。



(a) idle-speed 模式



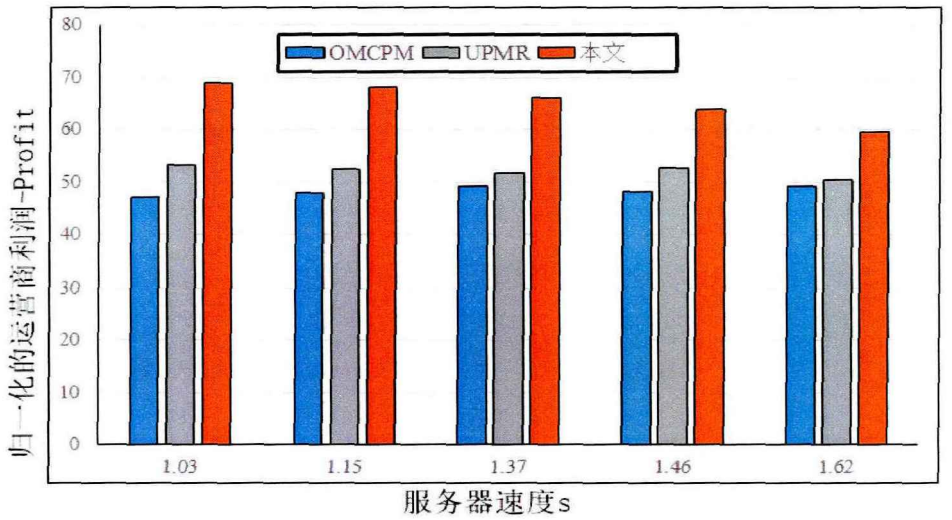
(b) constant-speed 模式

图 5-7 两种模式下的最优 M 与 s

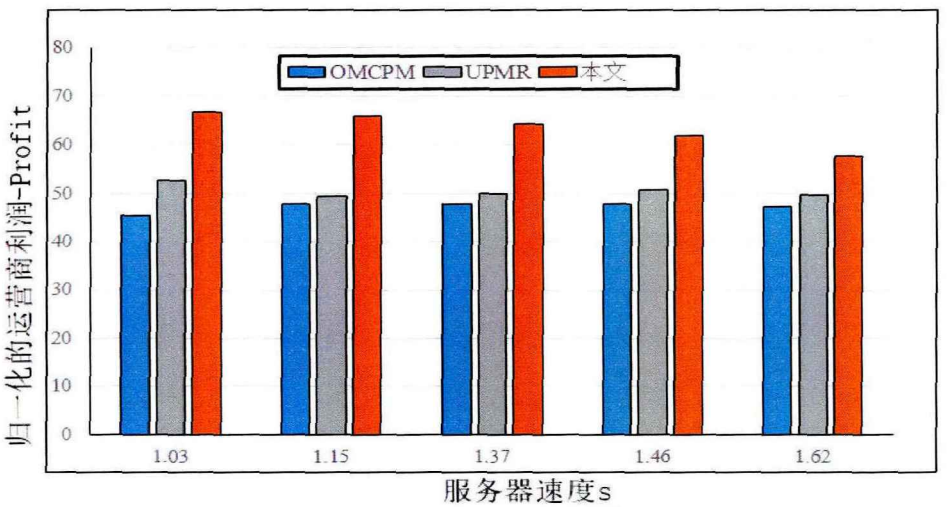
5.2.4 与已有相关研究的对比

为了突出本文所构建的利润最大化模型的优越性，本节将与已有的两种定价方案(optimal multiserver configuration for profit maximization，OMCPM)及(usage-based pricing with monetary reward，UPMR)进行关于最大利润的对比。除了采用如表 5-1 至表 5-3 所示的实验参数外，为了降低实验误差实现对比的公平性，本次对比实验有两组：服务请求到达率$\lambda_2 = 16.9$，工作服务器数量为$M = 17$以及服务请求到达率$\lambda_2 = 12.55$，工作服务器数量为$M = 18$。实验对比基于 idle-speed 和 constant-speed 两种服务器工作模式，将不同服务器运行速度下所得的最大利润(Profit)与已有方案进行对比。

图 5-8 展示了$\lambda_2 = 16.9$、$M = 17$时，在 idle-speed 及 constant-speed 两种模式下云服务运营商单位时间内所获最大利润的对比图。图 5-9 展示了$\lambda_2 = 12.55$、$M = 18$时，在 idle-speed 及 constant-speed 两种模式下云服务运营商单位时间内所获最大利润的对比图。横轴表示处理器的运行速度，纵轴表示利润(Profit)，为方便比较数据单位已做好归一化。从图中可以很清楚的看出，在各最优的处理器运行速度下，本文提出的定价策略均优于上述两种已有方案。

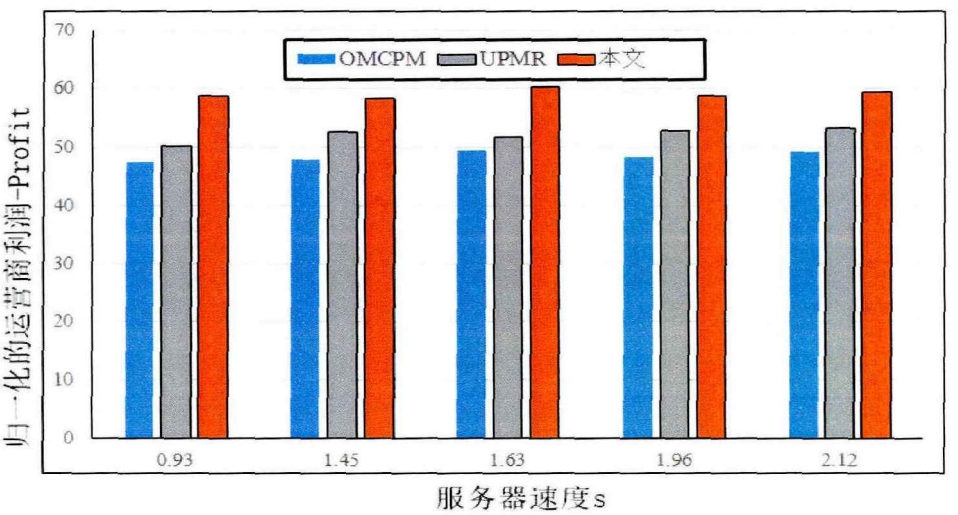


(a) idle-speed 模式

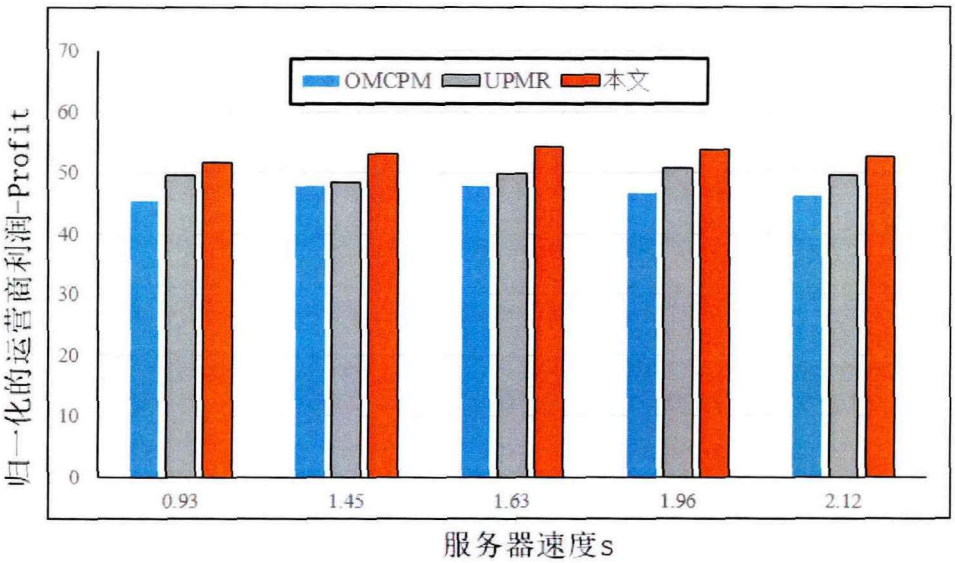


(b) constant-speed 模式

图 5-8 $\lambda_2 = 16.9$、M = 17时的最大利润对比图



(a) idle-speed 模式



(b) constant-speed 模式

图 5-9 $\lambda_2 = 12.55$、M = 18时的最大利润对比图

5.3 本章小结

本章主要是对本文构建的利润最大化模型进行基于实验数据的分析。从实验结果及相应分析中，不难看出本文提出的基于用户感知价值的云服务定价模型满足用户的需求定律。在实验中，本章详细展示了 idle-speed 与 constant-speed 两种模式下，不同工作负载及服务定价对云服务运营商收入及利润的影响。此外，除了获取最优的服务器数量与运行速度外，基于本文构建的利润模型，本章还将所得的最大利润与已有的研究在相同实验参数下进行对比，结果显示本文的利润模型能获得较高的利润。

第六章 总结与展望

6.1 本文总结

本文的目的是在云计算环境中，在保证用户服务满意度的前提下，获取最优的云服务定价以及最优的多服务器系统配置方案，使得云服务运营商基于该方案能获得最大的利润，即一种获取基于用户需求定律的最优服务定价和最优系统运行配置的利润最大化方案。

在探索该方案的过程中，本文首先对相关问题进行了建模。既然需要获取最优的云服务定价，那么需要一个合适的服务定价模型。与已有的有关定价的研究工作不同，本文基于用户的需求定律，构建了一个能反映用户在某个服务定价下需求分布的定价模型。然后本文根据不同用户对云服务的延迟敏感度不同这一实际情况，构建了一个基于延迟敏感度的返现奖励(Monetary Reward，MR)模型。该 MR 奖励模型根据用户设定的服务允许延迟时间，给用户相应的返现费用作为奖励。这一模型主要是增强用户的自主性(是否允许延迟由用户自己设定)，同时提高用户的服务满意度。对于云服务运营商，本文对其多服务器系统进行了建模，构建了多服务器系统模型。然后构建了云服务运营商的收入、耗费模型；其中耗费模型主要包括：电费、租赁费、对用户的 MR 奖励、低服务质量时的收费惩罚。最后基于上述模型，构建了如公式(3.1)所示的利润最大化问题。在构建利润最大化的模型中，还考虑了如下约束条件：服务请求的数量，服务的紧急程度，应用环境中的工作负载，服务质量 (QoS)，用户的服务满意度，服务等级协议(SLA)。本文设计了两个算法来求解利润最大化问题的数学表达式以获取最优解。

除了构建基于用户感知价值符合需求定律的利润最大化模型之外，本文还构建了一个动态自适应闭环控制系统模型。与其它单纯做服务定价的研究文献不同，本文构建的动态自适应模型，能够根据外界条件的改变而动态地调整服务定价。一旦外界某个条件改变（例如电费，租赁费等等），系统都需要动态地做出改变，重新获取最优的方案。本文同样设计了一个动态自适应算法用来实现上述模型。

从用户体验角度考虑，本文提出的 MR 返现奖励机制能保证用户服务满意度；基于响应时间约束的服务质量惩罚方案，遵循服务等级协议 (SLA)。从云服务运营商的角度，本文构建的利润模型能够使其利润最大化；由于构建的 MR 模型能够允许用户设定其服务允许的延迟时长，因此，该方案可以使得运营商能够弹性调度用户服务请求，并给运营商维护整个系统运行带来更大的灵活性。

6.2 未来工作方向

云计算作为一个商业运营模式，获得最大的利润是云服务运营商的首要目标。对此已经有无数的科研工作者进行过利润最大化方案的研究。本文构建的收费模型是一种基于使用量的收费模式。在整个收费周期内我们假定云服务的价格保持不变，但是在另一个收费周期内可能会因外界一些参数的改变，云服务的定价也会改变。基于此，未来的一个研究工作点是：构建一个线上的实时定价及收费模型。类似实时电价模型，不同服务请求在不同时间段或者高峰低谷期的收费不同，而不再针对整个收费周期。

本文的研究工作将用户按照其对延迟的敏感度不同，分为弹性用户和非弹性用户。因此，允许不同延迟时长的服务请求对云服务运营商的调度压力也不同。在研究过程中，本文只是保证用户在其最大允许延迟时长到达之前尽可能得到调度，采用的是 FCFS 策略，并没有考虑更先进的调度技术。此外，为方便研究，本文的多服务器系统假设都是同构的。所以，未来另一个研究方向是：基于本文的利润模型，考虑更好的调度技术问题。该调度问题涉及不同的异构虚拟机(VM)之间服务请求的分配，以及动态电压频率调整(DVFS)等调度技术方案。通过 DVFS 方案来获得能耗和多服务器系统负载之间的均衡。

除了上述可研究方向之外，未来的研究工作还可以考虑对服务质量保证的优化问题，侧重于用户的服务满意度；研究基于不同数据中心的不同服务的收费模型，该过程中可能涉及到不同数据中心之间的负载均衡及调度问题。因此，本文未来可研究方向比较宽广，能做的研究工作较多。

附录一 作者攻读硕士学位期间发表的科研成果

【专利】

[1]. 一种在可再生供能的云计算系统中基于合作博弈的能量分配方法，201510725103.9。

[2]. 一种云计算环境中基于用户感知价值的最优服务定价方法，201710009301.4

附录二 本文相关性质的证明与公式计算

A 关于需求分布性质的证明

A-1 证明公式(2.10)中的负二项分布满足性质 1

证明：公式(2.10)中，函数$P_\omega[m]$关于ω的一阶导数为：

$$\begin{aligned}P'_\omega[m] &= \frac{d}{d\omega}P_\omega[m]\\ &= \frac{d\Pi}{d\omega}\frac{d}{d\Pi}\binom{m+\alpha-1}{m}[1-\Pi]^m\Pi^\alpha\\ &= \frac{d\Pi}{d\omega}\binom{m+\alpha-1}{m}\{\alpha\Pi^{\alpha-1}[1-\Pi]^m - m[1-\Pi]^{m-1}\Pi^\alpha\}\\ &= \frac{d\Pi}{d\omega}\binom{m+\alpha-1}{m}[1-\Pi]^m\Pi^\alpha\left\{\frac{E[m]-m}{1-\Pi}\right\}\\ &= \frac{d\Pi}{d\omega}P_\omega[m]\left\{\frac{E[m]-m}{1-\Pi}\right\}\end{aligned}$$

其中，$\Pi = [1+\beta t\{1-F(\omega)\}]^{-1}$。

很容易看出Π的一阶导数是一个非负数：

$$\frac{\mathrm{d}\Pi}{\mathrm{d}\omega} = f(\omega)\beta t\{1+\beta t[1-F(\omega)]\}^{-2} \geq 0, \qquad 0 \leq \omega \leq \infty$$

因此，当$m \in (0, \infty)$时有：

$$\begin{aligned}\sum_{m=0}^{\infty}P'_\omega[m] &= \frac{\mathrm{d}\Pi}{\mathrm{d}\omega}\sum_{m=0}^{\infty}P_\omega[m]\left\{\frac{E[m]-m}{1-\Pi}\right\}\\ &= \frac{\mathrm{d}\Pi}{\mathrm{d}\omega}\{\frac{E[m]-E[m]}{1-\Pi}\}\\ &= 0\ .\end{aligned}$$

当m取有限的值时，即$m \in (0, c)$时总是正数：

$$\sum_{m=0}^{c}P'_\omega[m] > 0, 0 \leq c < \infty\ .$$

因此，由公式(2.10)中负二项分布所表示的用户需求分布满足性质 1。

A-2 证明公式(2.10)中的负二项分布满足性质 2

证明：由于任何累计分布函数都具有性质$\lim_{\omega\to\infty} F(\omega) \le 1$，且公式(2.10)的期望为$E_\omega[m] = \alpha\beta t\big(1 - F(\omega)\big)$。显然，$\lim_{\omega\to\infty} \omega\, E[m|\omega] = 0$。

因此，由公式(2.10)中负二项分布所表示的用户需求分布满足**性质 2**。

B 算法 1 中相关定理的证明

B-1 定理 1 的证明

考虑到$\lfloor D_i \rfloor$在θ上是非递减，定理 1 成立当且仅当：

$$\lfloor D_i \rfloor|_{\theta=U_b-\epsilon} = D_i^{L_b},\ \forall i \in I. \tag{B.1}$$

我们分两种情况证明上式(B.1)成立：$L_b \ge \max_i\{(D_{max}+1)\kappa_i\}$与$L_b < \max_i\{(D_{max}+1)\kappa_i\}$。

a. 若$L_b \ge \max_i\{(D_{max}+1)\kappa_i\}$:

根据公式(3.9)可得：$U_b = \infty$;

根据公式(3.8)可得：$\mathrm{D}_\mathrm{i}^{L_b} = D_{max}, \forall i \in I$;

根据公式(2.16)可得：$\lfloor D_i \rfloor|_{\theta=U_b-\epsilon} = D_{max}, \forall i \in I$;

因此，当$L_b \ge \max_i\{(D_{max}+1)\kappa_i\}$时，定理 1 成立。

b. 若$L_b < \max_i\{(D_{max}+1)\kappa_i\}$:

此时，有$U_b = min_{i\in\Theta}\{\big(D_i^{L_b}+2\big)\kappa_i\}$;

这种情况下，我们分如下步骤证明：

1. 式子$U_b = min_{i\in\Theta}\{\big(D_i^{L_b}+2\big)\kappa_i\}$，对于$\forall i \in \Theta$，有：

$$U_b \le \big(D_i^{L_b}+2\big)\kappa_i\}, \forall i \in \Theta\ , \tag{B.2}$$

将上式(B.2)带入公式(2.16)，可得：

$$\lfloor D_i \rfloor|_{\theta=U_b-\epsilon} \le \lfloor \max(\min(D_i^{L_b}+1-\epsilon, D_{max}),0) \rfloor\ , \tag{B.3}$$

在公式(3.8)中，我们保证$D_i^{L_b} \geq 0$。因此有$\min\left(D_i^{L_b}+1-\epsilon\right) \geq 0$, 联合上式(B.3)，可得：

$$\begin{aligned} \lfloor D_i \rfloor|_{\theta=U_b-\epsilon} &\leq \left\lfloor \min\left(D_i^{L_b}+1-\epsilon, D_{max}\right) \right\rfloor \\ &\leq \left\lfloor D_i^{L_b}+1-\epsilon \right\rfloor \end{aligned} , \tag{B.4}$$

由公式(3.8)可知，$D_i^{L_b}$是整型数值，因此有：$D_i^{L_b} = D_{max}$。此时，

$$\left\lfloor D_i^{L_b}+1-\epsilon \right\rfloor = D_i^{L_b} , \tag{B.5}$$

联合式子(B.4)与(B.5)，可得：

$$\lfloor D_i \rfloor|_{\theta=U_b-\epsilon} \leq D_i^{L_b}, \forall i \in \Theta , \tag{B.6}$$

2. 对于$\forall i \notin \Theta$，根据公式(3.8)(3.10)，可得：$D_i^{L_b} = D_{max}$, $\forall i \notin \Theta$。根据公式(2.16)，可得$\lfloor D_i \rfloor|_{\theta=U_b-\epsilon} \leq D_{max}, \forall i \in I$。因此：

$$\lfloor D_i \rfloor|_{\theta=U_b-\epsilon} \leq D_i^{L_b}, \forall i \notin \Theta , \tag{B.7}$$

联合公式(B.6)(B.7)，可得：

$$\lfloor D_i \rfloor|_{\theta=U_b-\epsilon} \leq D_i^{L_b}, \forall i \in I .$$

3. 不妨令

$$j = \arg\min_{i\in\Theta}\left\{\left(D_i^{L_b}+2\right)\kappa_i\right\}$$

由于当$L_b < \max_i\{(D_{max}+1)\kappa_i\}$时，有$U_b = min_{i\in\Theta}\{\left(D_i^{L_b}+2\right)\kappa_i\}$, 因此可得：

$$U_b = \left(D_j^{L_b}+2\right)\kappa_j , \tag{B.8}$$

所以对于$j \in \Theta$, 根据公式(3.10)可得：$L_b \in [0,\ (D_{max}+1)\kappa_j)$;

如果$L_b \in [0,\ 2\kappa_j)$，根据公式(3.8)可得：

$$D_j^{L_b} = 0 , \tag{B.9}$$

将公式(B.8)与(B.9)联合，得：$U_b = 2\kappa_j > L_b$；

如果$L_b \in [2\kappa_j, (D_{max}+1)\kappa_j)$，根据公式(3.8)可得：

$$D_j^{L_b} = \lfloor \frac{L_b}{\kappa_j} - 1 \rfloor \ , \tag{B.10}$$

将上式(B.10)与(B.8)联合，得：

$$U_b = \left(\left\lfloor \frac{L_b}{\kappa_j} - 1 \right\rfloor + 2 \right) \kappa_j > \left(\left\lfloor \frac{L_b}{\kappa_j} - 1 \right\rfloor + 1 \right) \kappa_j > L_b \ .$$

综上，有：$U_b > L_b$。同时由于$\lfloor D_i \rfloor$在θ上是非递减的，因此可得：

$$\lfloor D_i \rfloor|_{\theta = U_b - \epsilon} \geq D_i^{L_b}, \forall i \in I \ .$$

根据上述步骤 1 到步骤 3, 可知当$L_b < \max_i\{(D_{max}+1)\kappa_i\}$时，定理 1 成立。

综上，根据情况 a 与情况 b，可知定理 1 成立。

B-2 定理 2 的证明

由公式(3.13)表示的子优化问题中涉及到变量θ的部分只有关于奖励函数的定义，参见公式(2.28)。此外公式(2.28)表示的奖励函数中除了变量θ之外，并不涉及其它变量。同时，式(3.13)表示的子优化问题的约束条件中涉及到变量θ的只有公式(3.11)表示的约束条件。因此，式(3.13)表示的问题可以独立的分解为两个子优化问题，其中第一个是最小化对用户的奖励，即 Reward，由公式(2.27)(2.28)定义，满足的约束条件是(3.8)-(3.11)。在这种分解后的子问题中，θ的最优值是$\theta = L_b$，这是因为由公式(2.27)(2.28)定义的 Reward 函数是一个关于θ的非递减函数。所以，当$\theta = L_b$时，子优化问题(3.13)中的 Reward 部分可以取最小值。

综上，在分解后的每一个θ子域中，其最佳取值为$\theta = L_b$。

参考文献

[1] K. Hwang, J. Dongarra, and G. C. Fox, "Distributed and Cloud Computing," San Mateo, CA, USA: Elsevier/Morgan Kaufmann, 2012.

[2] J. Cao, K. Hwang, K. Li, and A. Y. Zomaya, "Optimal multiserver configuration for profit maximization in cloud computing," IEEE Trans. Parallel Distrib. Syst., vol. 24, no. 6, pp. 1087–1096, Jun. 2013.

[3] M. Armbrust, A. Fox, R. Griffith, A. Joseph, R. Katz, A. Konwinski, G. Lee, D. Patterson, A. Rabkin, and I. Stoica, "Above the clouds: A Berkeley view of cloud computing," Dept. Electrical Eng. Comput. Sci., vol. 28, pp.1–23, 2009,

[4] R. Buyya, C. S. Yeo, S. Venugopal, J. Broberg, and I. Brandic, "Cloud computing and emerging it platforms: Vision, hype, and reality for delivering computing as the 5th utility," Future Gener. Comput. Sys., vol. 25, no. 6, pp. 599–616, 2009.

[5] P. Mell and T. Grance, "The NIST definition of cloud computing," National Institute of Standards and Technology Inform. , vol. 15, 2011.

[6] D. Durkee, "Why Cloud Computing Will Never be Free," Comm. ACM, vol. 53, no. 5, pp. 62-69, 2010.

[7] Y. Lee, C. Wang, A. Zomaya, and B.B. Zhou, "Profit-Driven Service Request Scheduling in Clouds," Proc. 10th IEEE/ACM Int'l Conf. Cluster, Cloud and Grid Computing, pp. 15-24, 2010.

[8] J. Chen, C. Wang, B. Zhou, L. Sun, Y. Lee, and A. Zomaya, "Tradeoffs between profit and customer satisfaction for service provisioning in the cloud," in Proc.

[9] Y. Chun, "Optimal pricing and ordering policies for perishable commodities," European Journal of Operational Research 144, pp. 68-82, 2003.

[10] Y. Lee, C. Wang, A. Zomaya, and B. B. Zhou, "Profit-driven scheduling for cloud services with data access awareness," JPDC. vol. 72, no. 4, pp. 591–602, 2012.

[11] M. Ghamkhari and H. Mohsenian-Rad, "Energy and performance management of green data centers: A profit maximization approach," IEEE Trans. Smart Grid, vol. 4, no. 2, pp.1017–1025, 2013.

[12] A. Odlyzko, "Should flat-rate internet pricing continue," IT Professional, vol. 2, no. 5, pp. 48–51, 2000.

[13] G. Kesidis, A. Das, and G. de Veciana, "On flat-rate and usagebased pricing for tiered commodity internet services," in Proc. 42nd Annu. Conf. Inf. Sci. Syst., pp. 304–308, 2008.

[14] S. Shakkottai, R. Srikant, A. Ozdaglar, and D. Acemoglu, "The price of simplicity," IEEE J. Sel. Areas Commun., vol. 26, no. 7, pp. 1269–1276, Sep. 2008.

[15] H. Xu and B. Li, "Dynamic cloud pricing for revenue maximization," IEEE Trans. Cloud Comput., vol. 1, no. 2, pp. 158–171, Jul.-Dec. 2013.

[16] M. Mac_ias and J. Guitart, "A genetic model for pricing in cloud computing markets," in Proc. ACM Symp. Appl. Comput., 2011, pp. 113–118.

[17] Amazon EC2. (2014) [Online]. Available: http://aws.amazon.com.

[18] Amazon EC2 spot instances. (2014). Available: //aws.amazon.com/ec2/spot-instances.

[19] R. L. Hall, and C. J. Hitch, "Price theory and business behaviour," Oxford Economic Papers, no. 2, pp. 12–45, 1939.

[20] D. Kahneman, J. L. Knetsch, and R. Thaler, "Fairness as a constraint on profit seeking: Entitlements in the market," Am. Econ. Rev., vol. 76, pp. 728–741, 1986.

[21] J. Heo, D. Henriksson, X. Liu, and T. Abdelzaher, "Integrating adaptive components: An emerging challenge in performance adaptive systems and a server farm case-study," in Proc. IEEE 28th Int. Symp. Real-Time Syst. Symp., Dec. 2007, pp. 227–238.

[22] E. Pinheiro, R. Bianchini, E. V. Carrera, and T. Heath, "Dynamic cluster reconfiguration for power and performance," in Compilers and Operating Systems for Low Power. New York, NY, USA: Springer, pp. 75–93, 2003.

[23] X. Fan, W. Weber, and L. Barroso, "Power provisioning for a warehouse-sized computer," ACM SIGARCH Comput. Archit. News, vol. 35, no. 2, pp. 13–23, 2007.

[24] J. S. Chase, D. C. Anderson, P. N. Thakar, A. M. Vahdat, and R. P. Doyle, "Managing energy and server resources in hosting centers," ACM SIGOPS Oper. Syst. Rev., vol. 35, no. 5, pp. 103–116, 2001.

[25] M. Mazzucco and D. Dyachuk, "Optimizing cloud providers revenues via energy efficient server allocation," Sustainable Comput.: Inf. Syst., vol. 2, no. 1, pp. 1–12, 2012.

[26] C. Yeo, S. Venugopal, X. Chu, and R. Buyya, "Autonomic metered pricing for a utility computing service," Future Generation Computer Systems, vol. 26, no. 8, pp. 1368-1380, 2010.

[27] C. Joe-Wong and S. Sen. "Mathematical frameworks for pricing in the cloud: Revenue, fairness, and resource allocations," arXiv preprint arXiv:1212.0022 , 2012.

[28] J. Ouyang, A. Sahai and J. Pruyne, "A mechanism of specifying and determining pricing in utility computing environments," Business-Driven IT Management, BDIM'07, 2nd IEEE/IFIP International Workshop on, pp. 39-44, 2007.

[29] D. E. Irwin, L. E. Grit, and J. S. Chase, "Balancing risk and reward in a market-based task service," in Proc. 13th IEEE Int. Symp. High Perform. Distrib. Comput., pp. 160–169, 2004.

[30] Y. Zhan, M. Ghamkhari, D. Xu, "Extending Demand Response to Tenants in Cloud Data Centers via Non-intrusive Workload Flexibility Pricing," IEEE Transactions on Smart Grid, 2016.

[31] J. Chen, C. Wang, B. B. Zhou, L. Sun, Y. C. Lee, and A. Y. Zomaya, "Tradeoffs between profit and customer satisfaction for service provisioning in the cloud," in Proc. 20th Int. Symp. High Perform. Distrib. Comput., pp. 229–238, 2011.

[32] Z. Yang, R. Peterson, "Customer perceived value, satisfaction, and loyalty: The

role of switching costs," Psychology & Marketing, vol. 21 no. 10, pp. 799-822, 2004.

[33] G. McDougall, T. Levesqu, "Customer satisfaction with services: putting perceived value into the equation," Journal of services marketing, vol. 14, no. 5, pp. 392-410, 2000.

[34] 田志友, 蒋录全, 吴瑞明. 易腐商品最优订货批量与定价及其粒子群优化解[J]. 系统工程理论与实践, 2005, 25(3): 46-51.

[35] G. Gallego, G. Van Ryzin, "Optimal dynamic pricing of inventories with stochastic demand over finite horizons," Management Science, vol. 40, no. 8, pp. 999-1020, 1994.

[36] S. Karlin, C. Carr, "Prices and optimal inventory policy," Studies in Applied Probability and Management Science, pp. 159-172, 1962.

[37] A. Rajan, Rakesh, R. Steinberg, "Dynamic pricing and ordering decisions by a monopolist," Management Science, vol. 38, pp. 240–262, 1992.

[38] P. Pfeiffer, "Probability for Applications," Springer, New York, 1990.

[39] K. Li, "Optimal Load Distribution for Multiple Heterogeneous Blade Servers in a Cloud Computing Environment," Proc. 25th IEEE Int'l Parallel and Distributed Processing Symp. Workshops, pp. 943-952, 2011.

[40] B. Chun and D. Culler, "User-Centric Performance Analysis of Market-Based Cluster Batch Schedulers," Proc. Second IEEE/ACM Int'l Symp. Cluster Computing and the Grid, 2002.

[41] J. Sherwani, N. Ali, N. Lotia, Z. Hayat, and R. Buyya, "Libra: A Computational Economy-Based Job Scheduling System for Clusters,"Software - Practice and Experience, vol. 34, pp. 573-590, 2004

[42] C. Yeo and R. Buyya, "A Taxonomy of Market-Based Resource Management Systems for Utility-Driven Cluster Computing,"Software - Practice and Experience, vol. 36, pp. 1381-1419, 2006.

[43] K. Li, "Optimal Configuration of a Multicore Server Processor for Managing the

Power and Performance Tradeoff," J. Supercomputing, vol. 61, no. 1, pp. 189-214, 2012.

[44] H. Khazaei, J. Misic, and V. Misic, "Performance Analysis of Cloud Computing Centers Using M/G/m/m+r Queuing Systems," IEEE Trans. Parallel and Distributed Systems, vol. 23, no. 5, pp. 936-943, May 2012.

[45] L. Kleinrock, W. John and Sons, "Queueing Systems: Theory," vol. 1, p. 102, 1975.

[46] http://en.wikipedia.org/wiki/Service level agreement, 2012

[47] J. Little, S. Graves, "Little's Law," International Series in Operations Research and Management Science 115, pp. 81-100, 2008.

[48] A. Chandrakasan, S. Sheng, and R. Brodersen, "Low-Power CMOS Digital Design," IEEE Journal of Solid-State Circuits, vol. 27, no. 4, pp. 473-484, 1992.

[49] B. Zhai, D. Blaauw, D. Sylvester, and K. Flautner, "Theoretical and Prac-tical Limits of Dynamic Voltage Scaling," Proc. 41st Design Automation Conf, pp. 868-873, 2004.

[50] Enhanced Intel SpeedStep Technology for the Intel Pentium M Proces-sor," White Paper, Intel, Mar., 2004.

[51] J. Mei, K. Li, A. Ouyang and K. Li, "A profit maximization scheme with guaranteed quality of service in cloud computing," IEEE Transactions on Computers, vol. 64, no. 11, pp. 3064-3078, 2015.

[52] Condor. (2014) [Online]. Available: http://www.cs.wisc.edu/condor.

[53] T. Tannenbaum, D. Wright, K. Miller, and M. Livny, "Condor: A distributed job scheduler," in Beowulf Cluster Computing with Linux. Cambridge, MA, USA: MIT Press, pp. 307–350, 2002.

[54] J. Cao, K. Li, and I. Stojmenovic, "Optimal power allocation and load distribution for multiple heterogeneous multicore server processors across clouds and data centers," IEEE Trans. Comput., vol. 63, no. 1, pp. 45–58, 2014.

致 谢

时光荏苒，本毕业论文研究的完成，也预示着我的研究生生涯已接近尾声。经过三年的学习积累，不但了解到了实时系统及云计算研究领域的知识及发展前景，也实现了职业愿景，可谓收获颇丰。除了专业知识外，这三年里还结识了许多位可敬的老师和可爱的同学，他们给予我很多关于做人做事的指导和帮助。

感谢我的导师——魏同权副教授，是他的认真严谨，使我认识到对待任何工作都应持有端正的态度。魏老师是一个对待学术研究很认真严谨的人，比较看重学生的逻辑思维能力。读研的三年里，我的逻辑思维能力得到了很大提高，从一盘散沙似的思维到如今能逻辑清晰表达自己的想法，这其中魏老师的指导是居功至伟。刚刚接触论文时，写作与表达能力不是很强，是魏老师不遗余力地指导我，总是循循善诱传授我写作的方法和技巧，使我在论文阅读和写作方面取得了一定的进步。在这里，对于他给予我的所有教诲表示由衷的感谢。

感谢515和604实验室的每一位同学：周俊龙师兄以远超同龄博士的水平向我们展示了一个优秀博士生的风采，同时感谢周博士对我论文的帮助；感谢女友常艳杰在我写作论文期间给我的帮助，帮我指出错误和语言表达问题，同时也感谢她生活上带给我的快乐，在我情绪低落时给我的鼓励；还感谢曾经一起学习的小伙伴们：杨雁峰、陈箭飞、汪俊马、刘云鹏、夏青青、汤俊、梁文彬、鞠芊蕾，我从你们身上学会了很多。

感谢我的家人，感谢爸爸、妈妈、爷爷、奶奶和弟弟，不管发生什么事情，他们总给我最大的支持，是我坚强的后盾。祝他们身体健康，幸福开心。我将在今后的工作、学习中加倍努力，以期能够用自己的技能和头脑成为一个对社会有用的人。

最后，还特别感谢本文的各位评审老师，感谢你们能在百忙之中抽出宝贵的时间来审阅我的毕业论文。由于本人学术水平有限，文中缺陷在所难免，还望各位专家老师批评指正。